# 01장 파이썬이란 무엇인가?

필자는 파이썬 프롬프트( `>>>` )를 처음 본 순간부터 지금까지 파이썬과 함께해 왔다. '프로그래밍은 어렵고 지루하다'라는 고정 관념을 가졌던 필자에게 파이썬은 큰 충격이었다. 여러분도 이 책을 통해 파이썬의 매력에 빠져 보길 바란다. 01장에서는 파이썬의 특징과 장단점을 알아보고, 프로그래밍 환경을 구축하는 방법을 배운다. 그리고 간단한 파이썬 프로그램도 직접 작성해 본다.

**목차**


- 01-1 파이썬이란?
- 01-2 파이썬의 특징
- 01-3 파이썬으로 무엇을 할 수 있을까?
- 01-4 파이썬 설치하기
- 01-5 파이썬 둘러보기
- 01-6 파이썬과 에디터

# 01-1 파이썬이란?

파이썬(Python)은 1991년 네덜란드의 귀도 반 로섬(Guido van Rossum)이 개발한 인터프리터 언어다. 귀도는 자신이 좋아하는 코미디 쇼 '몬티 파이썬의 날아다니는 서커스(Monty Python's Flying Circus)'에서 파이썬이라는 이름을 따왔다.

> *인터프리터 언어란 소스 코드를 한 줄씩 해석해 즉시 실행하고 결과를 바로 확인할 수 있는 언어를 말한다.*

파이썬의 사전적 의미는 '고대 신화에 나오는 파르나소스 산 동굴에 살던 큰 뱀'이다. 아폴로 신이 델파이에서 파이썬을 퇴치했다는 이야기가 전해진다. 파이썬 책 표지와 아이콘에 뱀 모양이 많은 이유가 바로 여기에 있다.




파이썬은 교육용뿐만 아니라 기업 실무에서도 널리 쓰인다. 구글(Google), 메타(Meta), 넷플릭스(Netflix), 유튜브(YouTube), 드롭박스(Dropbox), 스포티파이(Spotify) 등 세계적인 IT 기업이 파이썬을 핵심 기술로 활용한다. 특히 인공지능, 데이터 분석, 웹 개발 분야에서 많이 사용된다.

파이썬은 공동 작업과 유지 보수가 쉽고 편리하다. 이 때문에 다른 언어로 작성된 많은 프로그램이 파이썬으로 재작성되고 있다. 국내에서도 파이썬의 가치가 인정받아 사용자층이 넓어지고 있으며, 파이썬으로 프로그램을 개발하는 업체도 늘고 있다.

# 01-2 파이썬의 특징

모든 프로그래밍 언어는 저마다 장점이 있지만, 파이썬은 다른 언어에서 쉽게 찾아볼 수 없는 독특한 매력을 가지고 있다. 파이썬의 특징을 알면 왜 파이썬을 공부해야 하는지, 시간을 투자할 만한 가치가 있는지 판단할 수 있을 것이다.


- 파이썬은 인간다운 언어이다
- 문법이 쉬워 빠르게 배울 수 있다
- 무료이지만 강력하다
- 간결하다
- 프로그래밍을 즐기게 해 준다
- 개발 속도가 빠르다


## 파이썬은 인간다운 언어이다

프로그래밍이란 인간이 생각하는 것을 컴퓨터에 지시하는 행위이다. 파이썬은 사람이 생각하는 방식을 그대로 표현할 수 있는 언어이다. 따라서 파이썬을 사용하면 컴퓨터의 사고 체계에 맞추어 프로그래밍하려고 애쓸 필요가 없다. 프로그램을 구상하자마자 머릿속에서 생각한 대로 코드를 술술 써 내려갈 수 있다.

다음 소스 코드를 보면 쉽게 이해될 것이다.

```
if 4 in [1,2,3,4]: print("4가 있습니다")
```

이 소스 코드는 다음처럼 읽을 수 있다.

```
만약 4가 1, 2, 3, 4 중에 있으면 "4가 있습니다"를 출력한다.
```

프로그래밍을 모르더라도 직관적으로 무엇을 뜻하는지 알 수 있지 않은가? 마치 영어 문장을 읽는 듯한 느낌이 든다.

## 문법이 쉬워 빠르게 배울 수 있다

어려운 문법과 수많은 규칙에 둘러싸인 언어에서 벗어나고 싶지 않은가? 파이썬은 문법이 매우 쉽고 간결하며 사람의 사고 체계와 닮아 있다. 배우기 쉬운 언어, 활용하기 쉬운 언어가 가장 좋은 언어가 아닐까? 유명한 프로그래머인 에릭 레이먼드(Eric Raymond)는 파이썬을 공부한 지 단 하루 만에 자신이 원하는 프로그램을 작성할 수 있었다고 한다.

> *프로그래밍 경험이 조금이라도 있다면 파이썬의 자료형, 함수, 클래스 만드는 법, 라이브러리 및 내장 함수 사용 방법 등을 익히는 데 일주일이면 충분하다.*

## 무료이지만 강력하다

파이썬은 오픈 소스(open source)이며 무료 소프트웨어이다. 사용료 걱정 없이 언제 어디서든 파이썬을 내려받아 사용할 수 있다.

> *오픈 소스란 누구나 자유롭게 사용하고 수정할 수 있도록 소스 코드를 공개한 소프트웨어를 말한다.*

또한 만들고자 하는 프로그램의 대부분을 파이썬으로 만들 수 있다. 물론 시스템 프로그래밍이나 하드웨어 제어와 같은 매우 복잡하고 반복 연산이 많은 프로그램은 파이썬과 어울리지 않는다. 하지만 파이썬은 이러한 약점을 극복할 수 있도록 다른 언어로 만든 프로그램을 포함시킬 수 있다.

'파이썬과 C는 찰떡궁합'이라는 말이 있다. 프로그램의 전반적인 뼈대는 파이썬으로 만들고, 빠른 실행 속도가 필요한 부분은 C로 만들어서 파이썬 프로그램 안에 포함시킬 수 있기 때문이다. 실제로 파이썬 라이브러리 중에는 파이썬만으로 제작된 것도 많지만 C로 만든 것도 많다. C로 만든 것은 대부분 속도가 빠르다.

> *파이썬 라이브러리는 파이썬 프로그램을 작성할 때 불러와 사용할 수 있도록 미리 만들어 놓은 파이썬 파일의 모음이다.*

## 간결하다

파이썬의 창시자 귀도 반 로섬(Guido van Rossum)은 파이썬을 의도적으로 간결하게 만들었다. 펄(Perl)과 같은 프로그래밍 언어가 100가지 방법으로 1가지 일을 처리할 수 있다면, 파이썬은 가장 좋은 방법 1가지만 사용하는 것을 선호한다. 이 간결함의 철학은 파이썬 문법에도 그대로 녹아 있어 파이썬 프로그래밍을 하는 사람들은 잘 정리된 소스 코드를 볼 수 있다. 덕분에 다른 사람이 작업한 소스 코드를 이해하기 쉽고 공동 작업과 유지 보수가 편리하다.

다음은 파이썬 프로그램의 간결함을 보여 주는 예제이다. 지금은 이 소스 코드를 굳이 이해하려 하지 않아도 된다.

```
languages = ['python', 'perl', 'c', 'java']

for lang in languages:
    if lang in ['python', 'perl']:
        print("%6s need interpreter" % lang)
    elif lang in ['c', 'java']:
        print("%6s need compiler" % lang)
    else:
        print("should not reach here")
```

프로그래밍 언어를 판별하여 그에 맞는 문장을 출력하는 파이썬 프로그램 예제이다. 다른 언어에서 늘 보게 되는 단락을 구분하는 괄호({})가 없고, 줄을 잘 맞춘 코드라는 것을 알 수 있다. 파이썬 프로그램은 줄을 맞추지 않으면 실행되지 않는다. 코드를 예쁘게 작성하려고 줄을 맞추는 것이 아니라 프로그램이 실행되려면 반드시 줄을 맞추어야 한다. 이렇게 줄을 맞추어 코드를 작성하면 가독성이 좋아진다.

> *코드의 줄을 맞추는 것을 '들여쓰기'라고 한다. 파이썬은 들여쓰기를 하지 않으면 프로그램이 실행되지 않는다.*

## 프로그래밍을 즐기게 해 준다

파이썬은 부수적인 개념이나 제한 사항에 신경 쓸 필요 없이 만들고자 하는 기능에만 집중할 수 있게 해 준다. 예를 들어 화면에 "Hello World"를 출력하려면 파이썬은 `print("Hello World")` 한 줄이면 되지만, 다른 언어에서는 여러 줄의 코드가 필요한 경우가 많다. 파이썬을 배우고 나면 다른 언어로 프로그래밍하는 것이 지루하게 느껴질지도 모른다.

## 개발 속도가 빠르다

마지막으로 재미있는 문장으로 파이썬의 특징을 마무리하겠다.

***"Life is too short, You need python." (인생은 너무 짧으니 파이썬이 필요해.)***

엄청나게 빠른 파이썬의 개발 속도 때문에 유행처럼 퍼진 말이다. 이 위트 있는 문장은 이 책에서 계속 예제로 사용할 것이다.

# 01-3 파이썬으로 무엇을 할 수 있을까?

프로그래밍 언어를 좋은 언어와 나쁜 언어로 구별할 수 있을까? 사실 이런 구별은 무의미하다. 어떤 언어든 강점과 약점이 있기 때문이다. 따라서 각 언어가 어떤 일에 효율적인지 아는 것이 프로그래머의 생산성을 크게 높인다.

그렇다면 파이썬으로 하기에 적합한 일과 그렇지 않은 일은 무엇일까? 파이썬을 배우기 전에 간단히 알아보자.


- 파이썬으로 할 수 있는 일
  - 웹 프로그래밍
  - 인공지능과 머신러닝
  - 수치 연산 프로그래밍
  - 데이터 분석
  - 데이터베이스 프로그래밍
  - 시스템 유틸리티 제작
  - GUI 프로그래밍
  - C/C++와 결합
  - 사물 인터넷
- 파이썬으로 할 수 없는 일
  - 시스템과 밀접한 프로그래밍 영역
  - 모바일 프로그래밍


## 파이썬으로 할 수 있는 일

파이썬으로 할 수 있는 일은 매우 많다. 대부분의 프로그래밍 언어가 하는 일을 더 쉽고 깔끔하게 처리한다. 대표적인 몇 가지만 소개한다.

### 웹 프로그래밍

누구나 한 번쯤 구글 크롬이나 마이크로소프트 엣지 같은 웹 브라우저로 웹 서핑을 하면서 게시판이나 방명록에 글을 남겨 본 적이 있을 것이다. 이런 게시판이나 방명록을 '웹 프로그램'이라고 한다. 파이썬은 웹 프로그램을 만들기에 매우 적합하며, 실제로 파이썬으로 만든 웹 사이트는 셀 수 없이 많다.

## 인공지능과 머신러닝

인공지능(Artificial Intelligence) 분야의 발전으로 파이썬이 인기 언어로 떠올랐다. 인공지능은 기계가 인간처럼 생각하고 행동하게 하는 기술이다. 파이썬으로 자연어 처리, 음성 인식, 이미지 인식, 챗봇 개발 같은 인공지능 기술을 구현할 수 있다. 머신러닝(Machine Learning)은 인공지능의 한 분야로, 경험을 통해 자동으로 발전하는 컴퓨터 알고리즘을 연구한다. 파이썬에는 인공지능과 머신러닝을 쉽게 구현할 수 있는 다양한 라이브러리가 있으며, 대표적으로 사이킷런(scikit-learn), 텐서플로(TensorFlow), 파이토치(PyTorch), 케라스(Keras) 등이 있다. 이런 이름을 지금 외울 필요는 없고, 파이썬이 인공지능 분야에서 가장 널리 사용되는 언어라는 점만 기억하자.

## 수치 연산 프로그래밍

파이썬 자체는 C 같은 언어보다 수치 연산 속도가 느리다. 하지만 넘파이(NumPy)라는 수치 연산 모듈을 사용하면 이 문제를 해결할 수 있다. NumPy는 내부적으로 C로 작성되어 빠른 속도를 제공하면서도, 파이썬의 간결한 문법으로 사용할 수 있어 실무에서 널리 활용된다.




## 데이터 분석

파이썬은 데이터 분석에 이상적인 언어다. 넘파이(NumPy), 판다스(Pandas), 맷플롯립(Matplotlib) 등의 라이브러리로 데이터 처리, 통계 분석, 시각화를 손쉽게 할 수 있다. 데이터 분석에 특화된 R이라는 언어도 많이 사용되지만, 판다스가 등장한 이후 파이썬을 사용하는 경우가 점점 늘고 있다.




## 데이터베이스 프로그래밍

파이썬은 사이베이스(Sybase), 인포믹스(Informix), 오라클(Oracle), 마이에스큐엘(MySQL), 포스트그레스큐엘(PostgreSQL) 등의 데이터베이스에 접근하기 위한 도구를 제공한다.

이런 데이터베이스를 직접 사용하는 것 외에, 데이터베이스 없이도 간단하게 데이터를 저장하는 방법이 있다. 바로 피클(pickle)이라는 모듈이다. 피클은 파이썬 자료를 변형 없이 파일에 저장하거나 불러올 수 있다.

> *피클 사용법과 활용 방법은 '05-6 표준 라이브러리'를 참고하자.*

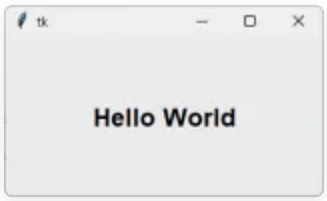
## 시스템 유틸리티 제작

파이썬은 운영체제(윈도우, 리눅스 등)의 시스템 명령어를 사용할 수 있는 각종 도구를 갖추고 있다. 이를 바탕으로 파일 관리, 로그 분석, 시스템 모니터링 등 여러 가지 시스템 유틸리티를 만들 수 있다. 실제로 시스템에서 사용 중인 서로 다른 유틸리티 프로그램을 하나로 합쳐 더 유용한 프로그램을 만들 수 있다.

> *유틸리티란 컴퓨터 사용에 도움을 주는 소프트웨어를 말한다.*

## GUI 프로그래밍

GUI(graphic user interface) 프로그래밍이란 화면에 윈도우 창을 만들고 그 창에 프로그램을 동작시킬 수 있는 메뉴나 버튼 등을 추가하는 것을 말한다. 파이썬은 GUI 프로그래밍을 위한 도구가 잘 갖춰져 있어 GUI 프로그램을 쉽게 만들 수 있다. 대표적인 예로 파이썬과 함께 설치되는 Tkinter(티케이인터)가 있다. Tkinter를 사용하면 단 5줄의 코드만으로 윈도우 창을 띄울 수 있다.



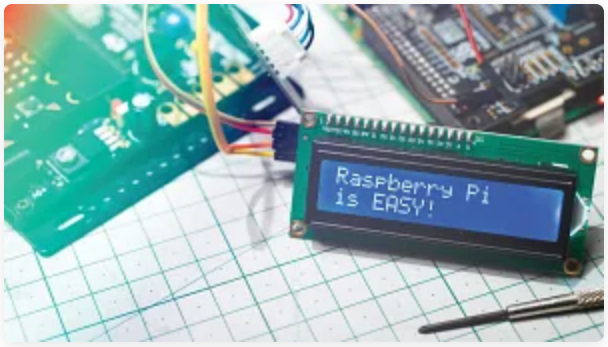

## C/C++와 결합

파이썬은 접착(glue) 언어라고도 부른다. 다른 언어와 결합해서 사용할 수 있기 때문이다. C나 C++로 만든 프로그램을 파이썬에서 사용할 수 있고, 파이썬으로 만든 프로그램도 C나 C++에서 사용할 수 있다. 예를 들어 속도가 중요한 핵심 부분은 C로 작성하고, 나머지 부분은 파이썬으로 편리하게 작성하는 식으로 두 언어의 장점을 결합할 수 있다.

## 사물 인터넷

파이썬은 사물 인터넷(Internet of Things) 분야에서도 활용도가 높다. 예를 들어 라즈베리파이(Raspberry Pi)는 리눅스 기반의 매우 작은 컴퓨터다. 라즈베리파이로 홈시어터나 작은 게임기 등 여러 가지 재미있는 것을 만들 수 있는데, 이때 파이썬으로

라즈베리파이를 제어한다. 라즈베리파이에 연결된 모터를 작동시키거나 LED에 불을 켜는 일을 파이썬으로 할 수 있다.




## 파이썬으로 할 수 없는 일

### 시스템과 밀접한 프로그래밍 영역

파이썬으로 리눅스 같은 운영체제, 장치 드라이버, 또는 엄청난 횟수의 반복과 연산이 필요한 프로그램을 만드는 것은 어렵다. 즉, 매우 빠른 속도가 필요하거나 하드웨어를 직접 제어해야 하는 프로그램에는 적합하지 않다. 이런 분야에서는 주로 C나 C++ 같은 언어를 사용한다.

### 모바일 프로그래밍

파이썬은 구글이 가장 많이 사용하는 언어지만, 파이썬으로 안드로이드 네이티브 앱(android native app)을 개발하는 것은 여전히 제한적이다. 네이티브 앱이란 특정 운영체제(안드로이드, iOS)에 맞춰 전용 언어로 만든 앱을 말한다. Kivy나 BeeWare 같은 프레임워크로 모바일 앱 개발이 가능하지만, 성능이나 네이티브 기능 활용 면에서는 한계가 있다. 아이폰 앱도 마찬가지로 직접적인 네이티브 개발보다는 하나의 코드로 여러 운영체제에서 동작하는 크로스 플랫폼 도구를 활용한 개발이 주를 이룬다.

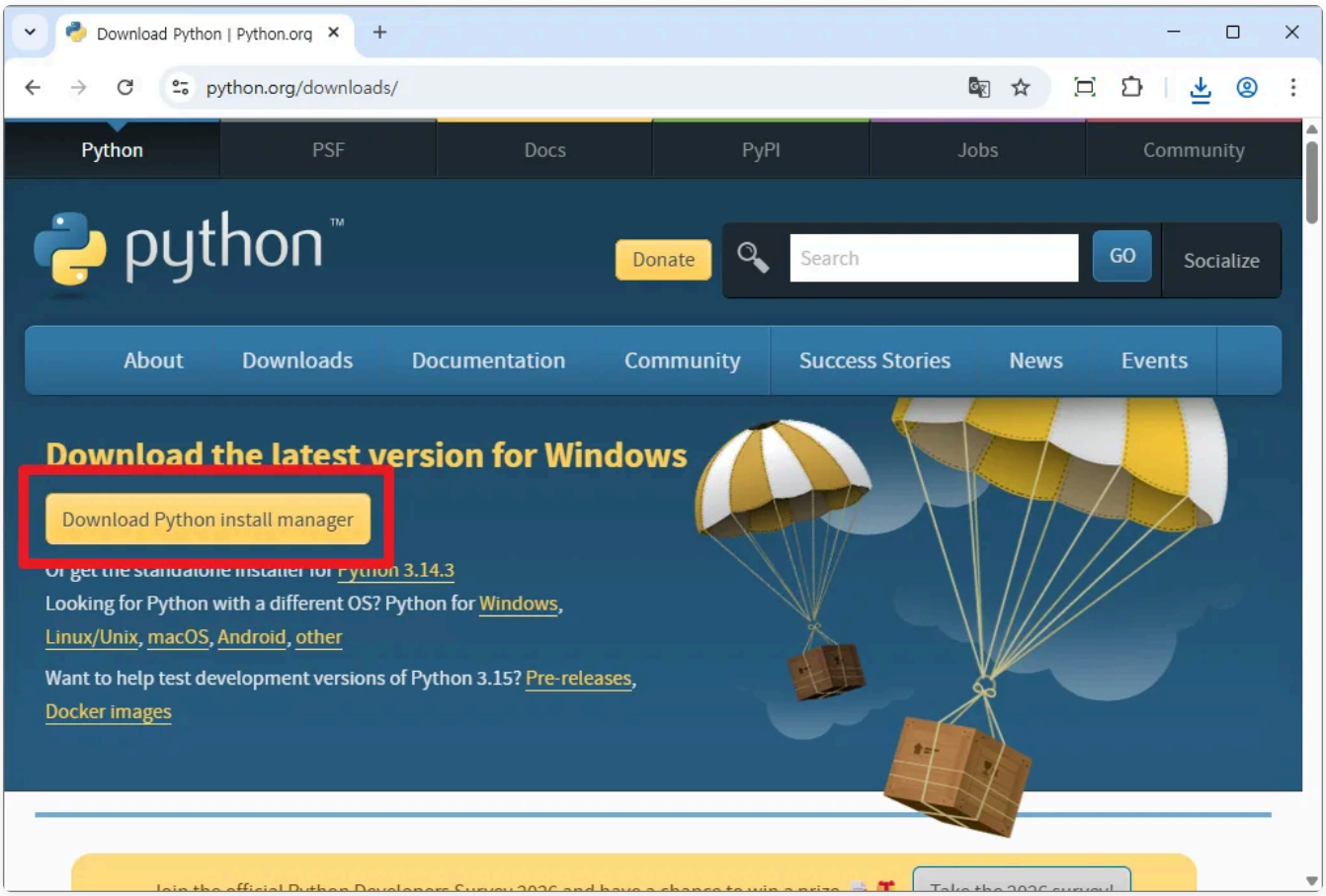
# 01-4 파이썬 설치하기

이제 실습을 위해 컴퓨터에 파이썬을 설치해 보자. 이 책에서는 윈도우와 맥 설치 방법만 다룬다. 다른 시스템을 사용한다면 파이썬 홈페이지(www.python.org)를 참고하자.

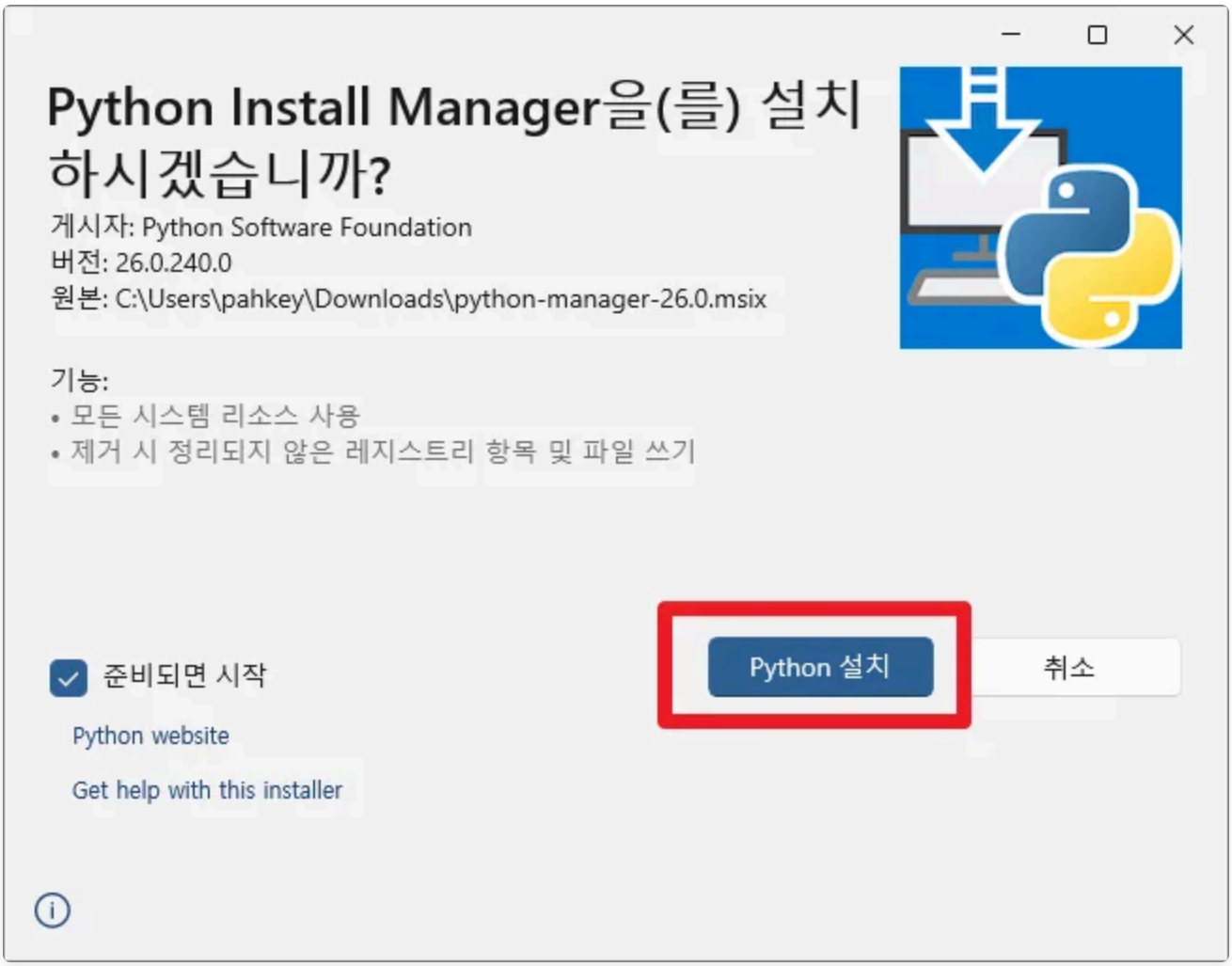

- 윈도우에서 파이썬 설치하기
- 맥에서 파이썬 설치하기


## 윈도우에서 파이썬 설치하기

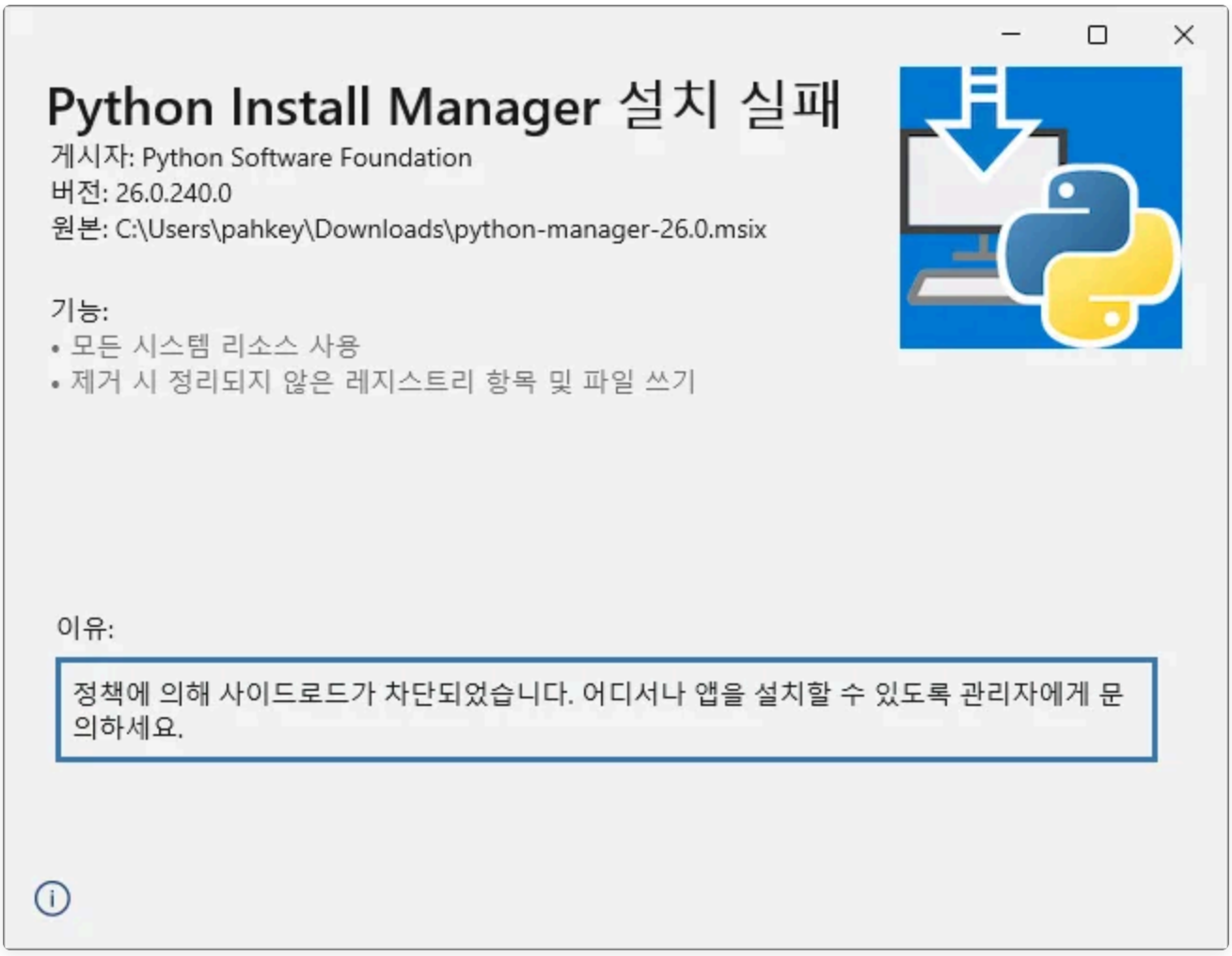
**1.** 파이썬 공식 홈페이지의 다운로드 페이지(www.python.org/downloads)에서 윈도우용 파이썬 설치 파일을 내려받자. [Download Python Install manager]를 선택하면 된다(2026년 3월 기준 최신 안정 버전은 3.14.3이다).



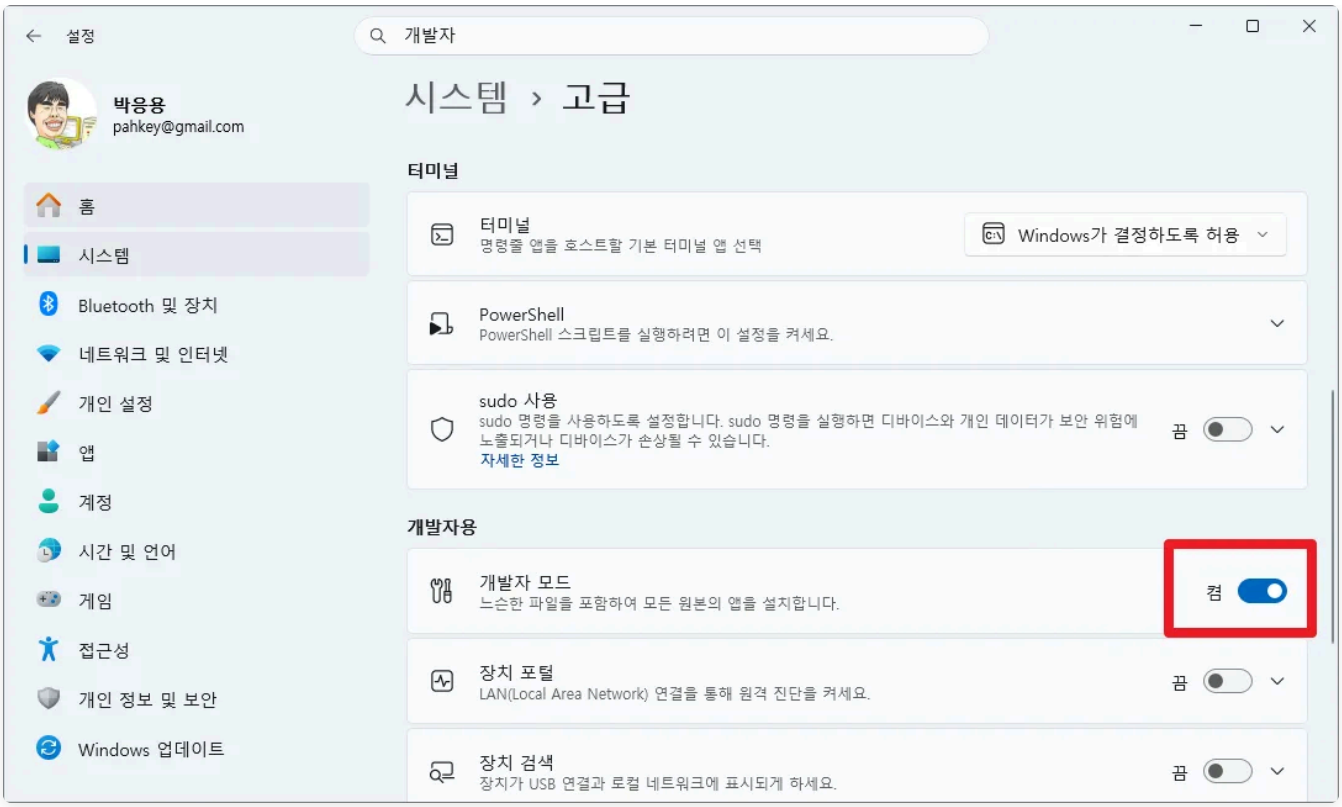

**2.** 다운로드 받은 파일(예: python-manager-26.0.msix)을 실행하여 [Python 설치]를 클릭하면 설치가 진행된다.



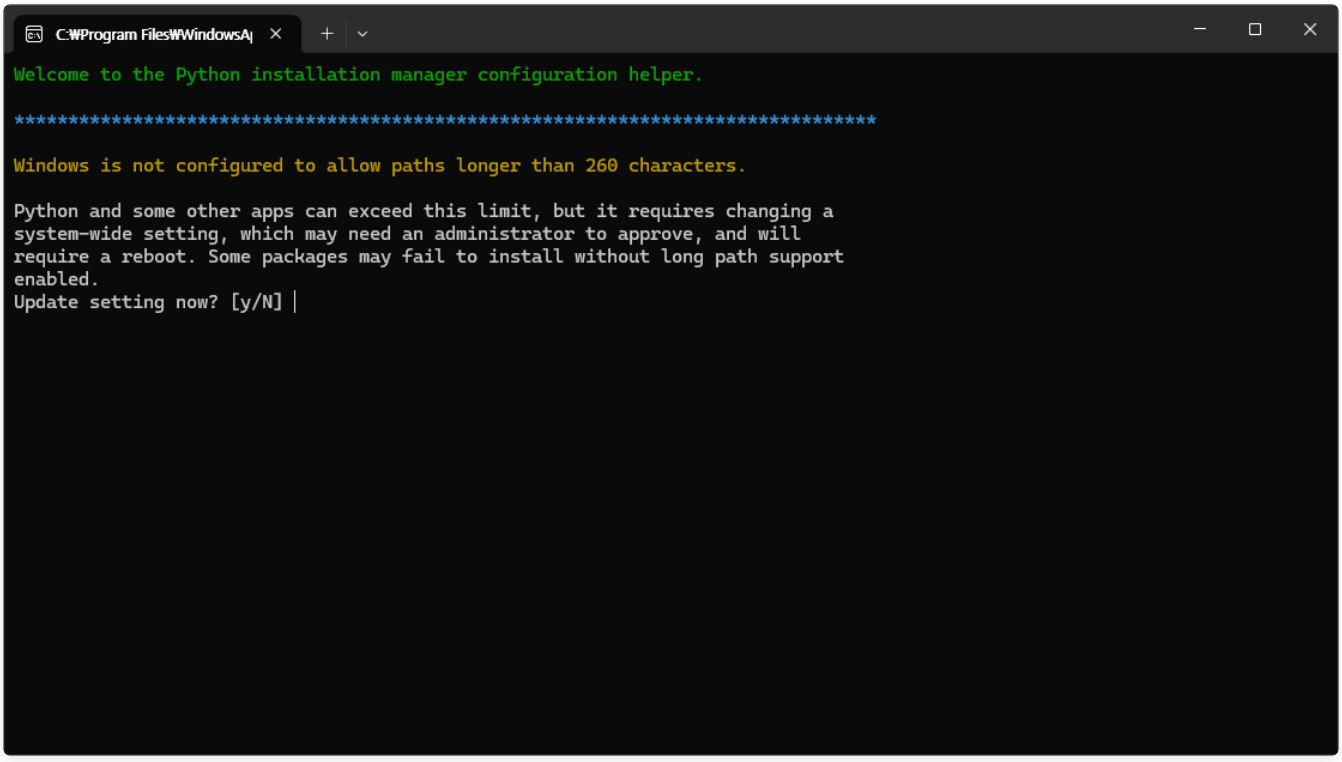

윈도우 환경에 따라서 설치 시 "정책에 의해 사이드로드가 차단되었습니다." 오류가 발생할 수 있다.




이런 오류가 발생하면 [윈도우 설정 > 시스템 > 고급] 메뉴에서 개발자 모드를 "켬"으로 수정한 후 다시 설치를 진행하자. 개발자 모드를 켜는 것은 파이썬 설치를 위해 필요한 설정이며, 컴퓨터에 문제를 일으키지 않으니 안심해도 된다.




오류없이 설치가 진행되면 다음 화면이 나타난다.




이 화면에서는 다음 질문들에 대한 답을 해야 한다.

1. Windows is not configured to allow paths longer than 260 characters. - 긴 경로를 허용할지 묻는 질문이다. (엔터 입력)

2. The global shortcuts directory is not configured. - 어느 위치에서든 파이썬을 실행할 수 있도록 경로를 설정할지 묻는 질문이다. (반드시 y를 입력)
3. You do not have the latest Python runtime. - 최신 파이썬을 설치할지 묻는 질문이다. (엔터 입력)

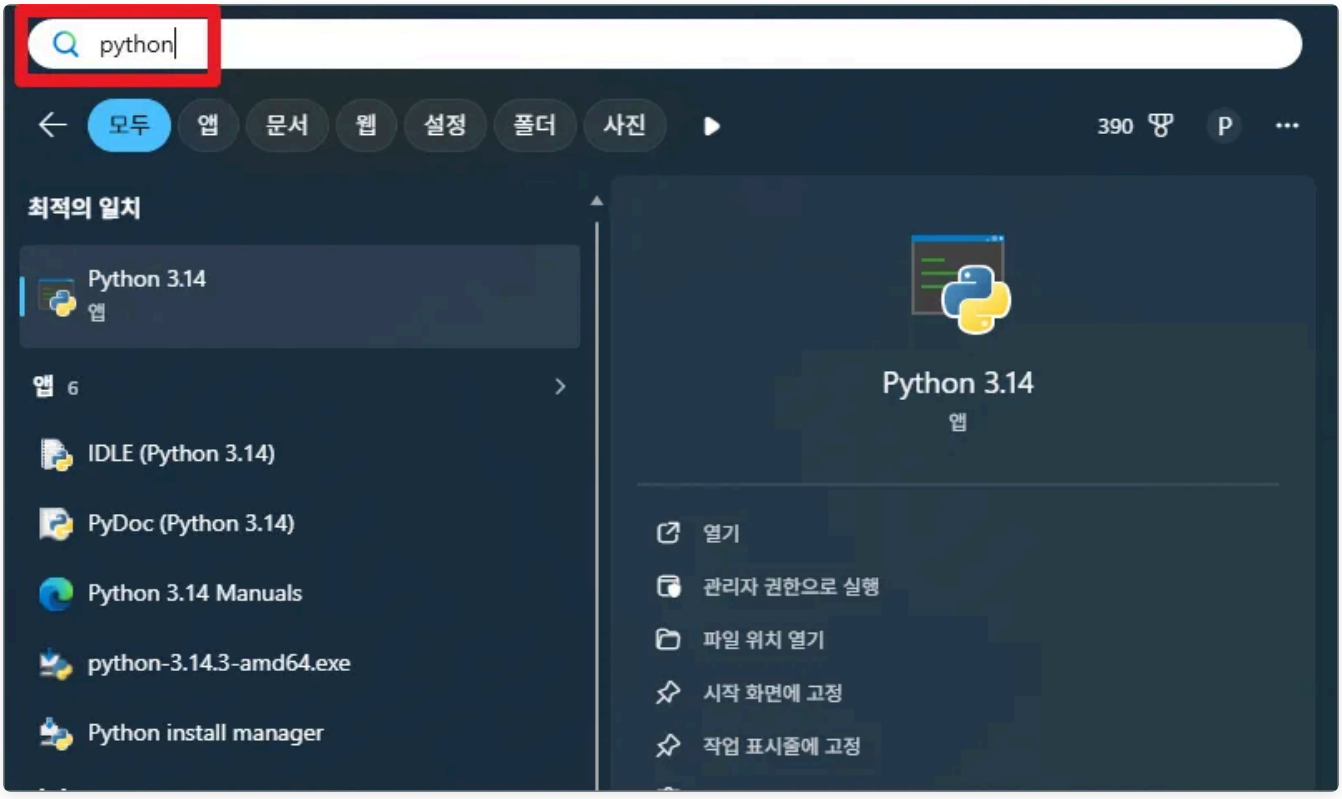
[중요] 2번째 항목은 반드시 "Y" 를 선택해야 한다. 이 옵션을 선택해야 어느 위치에서든 파이썬을 실행할 수 있다. 다음 질문에 반드시 "y"를 입력하고 엔터를 입력하자.

```
Add commands directory to your PATH now? [y/N]
```

> *이 옵션을 선택하지 않으면 실습 시 오류가 발생할 수 있다.*

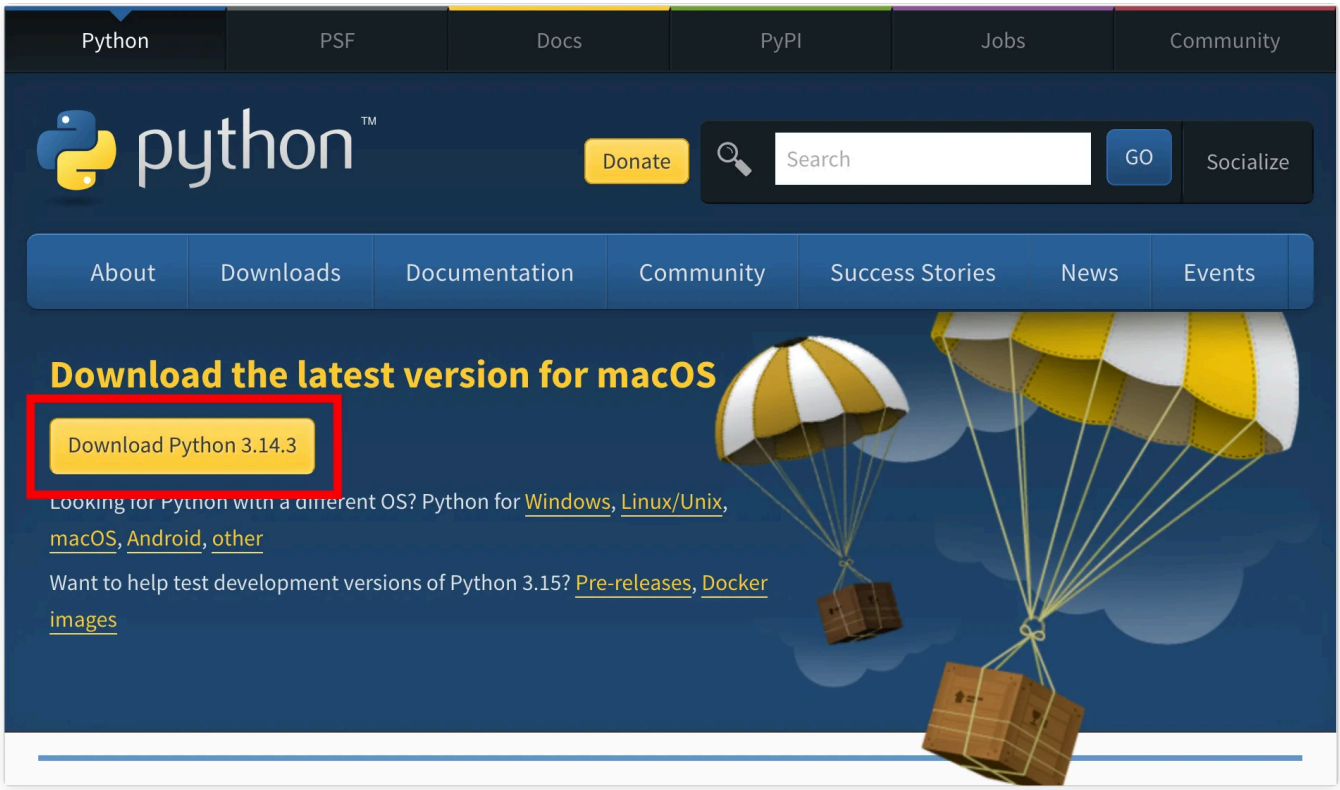
나머지 ◘질문에 대한 답은 모두 엔터를 입력하여 디폴트 값을 선택하면 된다.

**3.** 설치가 완료되면 [시작] 메뉴의 검색 창에서 python을 검색하여 실행 프로그램을 찾을 수 있다.




## 맥에서 파이썬 설치하기

**1.** 파이썬 공식 홈페이지(www.python.org)에서 [Downloads] 메뉴를 클릭하여 맥용 파이썬 설치 파일을 내려받자. 최신 안정 버전의 [Download Python 3.x.x] 버튼을 클릭하면 된다(2026년 3월 기준 최신 안정 버전은 3.14.3이다).




**2.** 내려받은 macOS용 설치 패키지 파일(예: `python-3.x.x-macos11.pkg`)을 실행하여 설치한다.

**3.** 설치가 완료되면 터미널에서 `python3 -V` 명령을 입력하여 파이썬이 제대로 설치되었는지 확인하자. 맥에서는 `python` 명령이 없거나 다른 버전이 실행될 수 있으므로 `python3` 명령을 사용해야 한다. 파이썬 버전이 출력되면 설치 성공이다.

> *터미널은 Spotlight(⌘ + Space)에서 "터미널"을 검색하여 실행할 수 있다.*

```
pahkey@mymac ~ % python3 -V
Python 3.x.x
```

> *`-V` 에서 V는 소문자가 아닌 대문자로 입력해야 한다.*

# 01-5 파이썬 둘러보기

파이썬이 어떻게 생겼는지 간단한 코드를 작성하며 알아보자. 전체 모습을 먼저 훑어보면 파이썬을 이해하는 데 큰 도움이 된다.

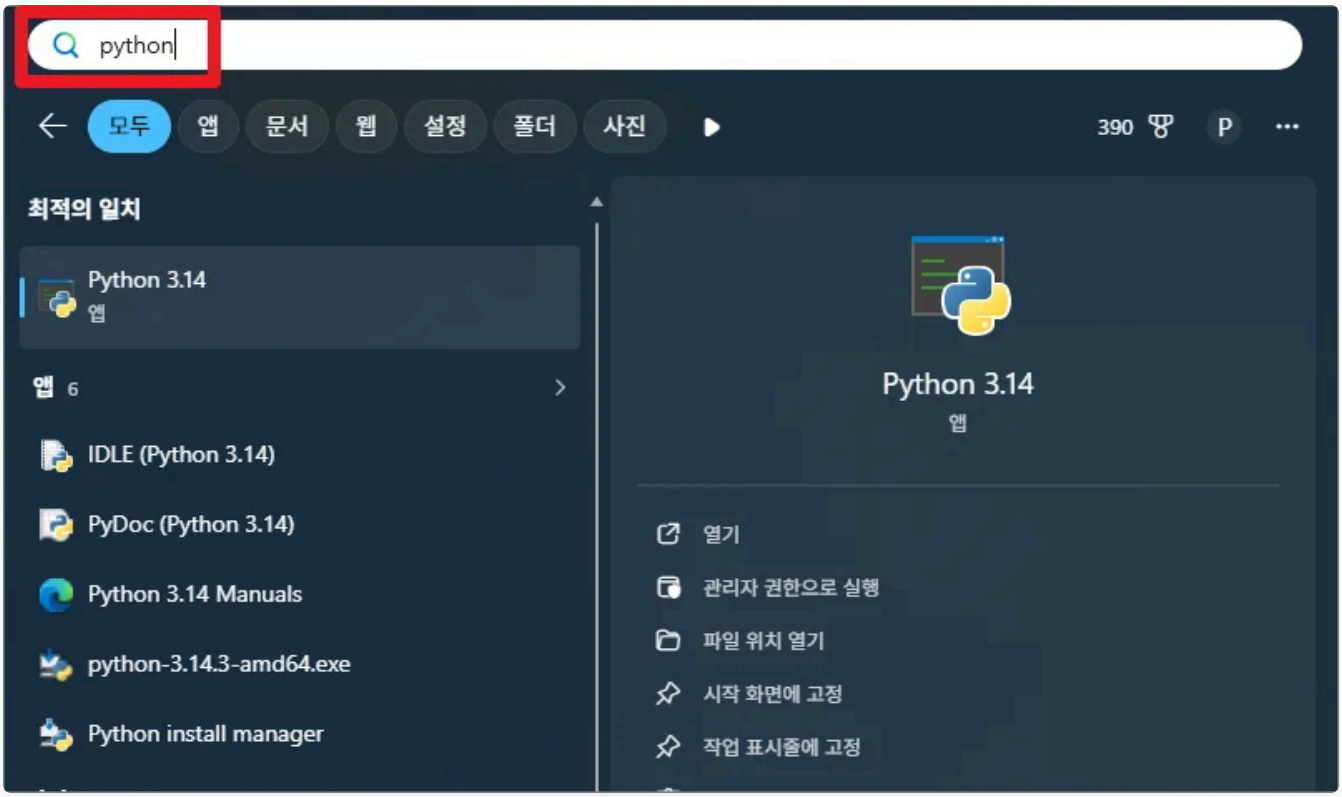
'백문이 불여일견, 백견이 불여일타'라고 했다. 직접 따라 해 보자.


- 파이썬 기초 실습 준비하기
- 파이썬 기초 문법 따라 해 보기
    - 사칙 연산
    - 변수에 숫자 대입하고 계산하기
    - 변수에 문자 대입하고 출력하기
    - 조건문 if
    - 반복문 for
    - 반복문 while
    - 함수


## 파이썬 기초 실습 준비하기

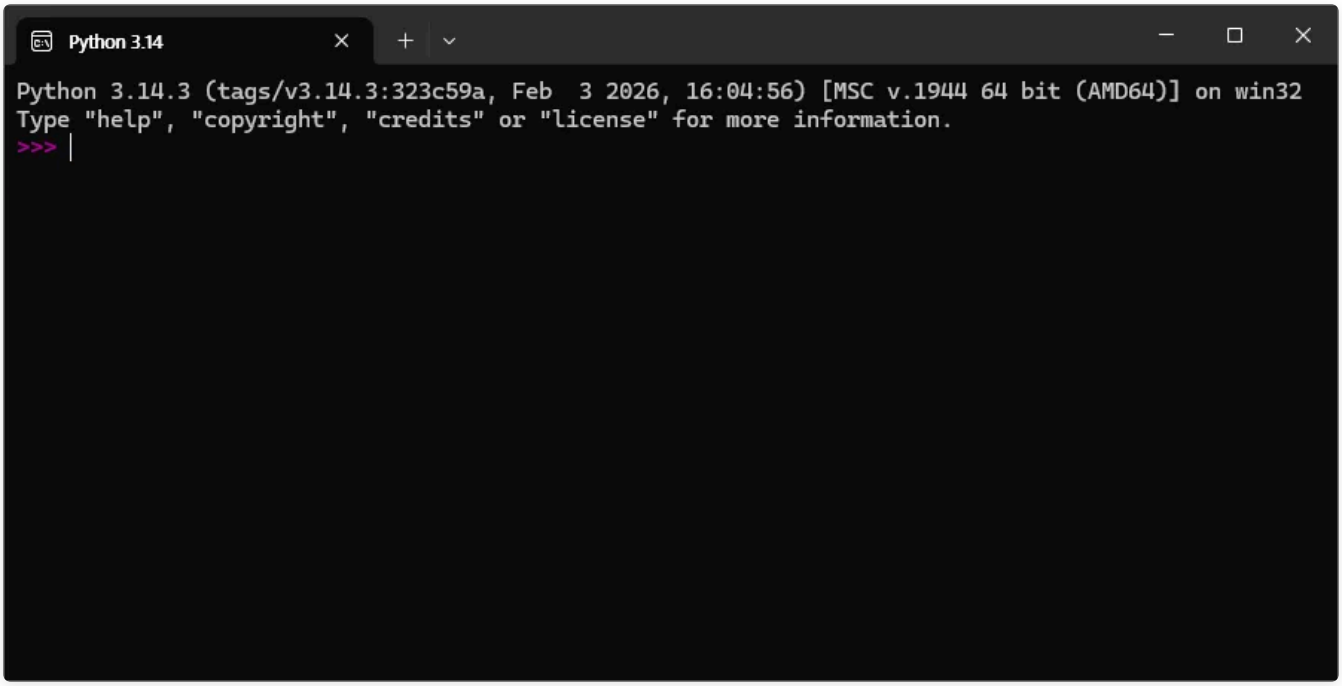
파이썬 프로그래밍 실습을 시작하기 전에 기초적인 것을 준비해 보자.

윈도우 사용자는 [시작] 메뉴에서 'Python 3.x'을 찾아 실행한다.




맥 사용자는 터미널(Spotlight에서 "터미널" 검색)을 열고 `python3` 을 입력하여 실행한다.

그러면 다음과 같은 화면이 나타난다.

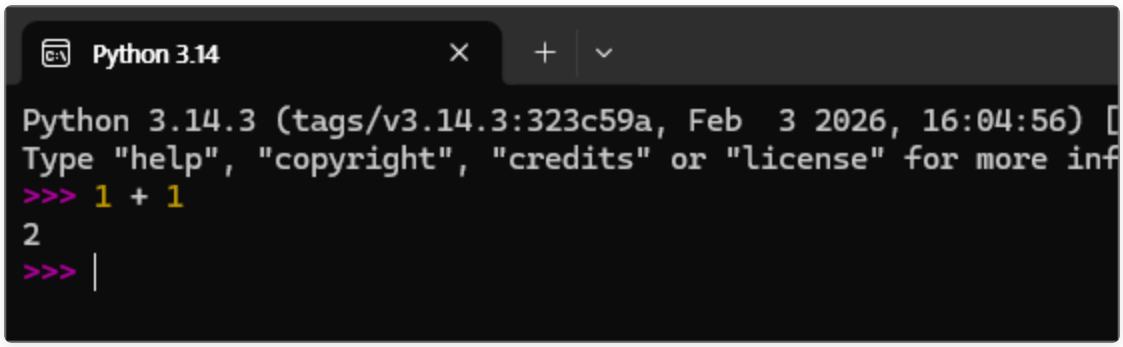



```
Python 3.x.x ( ... ) [MSC v.xxxx 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

이 창을 '파이썬 대화형 인터프리터'라고 부른다. 인터프리터란 사용자가 입력한 코드를 실행하는 환경을 뜻한다.

> - *파이썬 대화형 인터프리터를 간단히 '파이썬 셸(python shell)'이라고도 부른다.*
> - *파이썬 셸 창에서 3개의 꺾은 괄호( `>>>` )를 프롬프트(prompt)라고 부른다.*
> - *프롬프트( `>>>` )가 있는 곳이 입력하는 부분이다.*

그렇다면 '대화형'은 무슨 의미일까? 다음과 같이 간단한 수식을 입력해 보자.




```
>>> 1 + 1
2
```

입력에 따른 결괏값이 바로 출력된다. 이렇게 입출력이 번갈아 이어지는 것이 마치 대화하는 것처럼 느껴지기 때문에 파이썬 대화형 인터프리터라고 부른다. 대화형 인터프리터를 사용하면 결과를 바로 확인할 수 있어 학습에 효과적이다. 앞으로 파이썬의 기본적인 내용은 이 창을 사용해 실습한다.

대화형 인터프리터를 종료할 때는 `quit()` 또는 `exit()` 를 입력하면 된다.

```
>>> quit()
```

> *키보드 단축키로도 종료할 수 있다. 윈도우에서는 [ `Ctrl+Z` ]를 누른 후 [ `Enter` ], 맥/리눅스에서는 [ `Ctrl+D` ]를 누르면 된다.*

# 파이썬 기초 문법 따라 해 보기

이제 파이썬 대화형 인터프리터(파이썬 셸)를 실행하여 파이썬 기초 문법을 따라 해 보자. 여기에서 소개하는 내용은 뒤에서 자세하게 다룰 것이므로 지금 당장 이해되지 않아도 걱정하지 말자.

## 사칙 연산

1 더하기(+) 2는 3이라는 값을 출력해 보자. 계산기를 사용하듯이 +만 넣어 주면 된다.

```
>>> 1 + 2
3
```

나눗셈( `/` )과 곱셈( `*` ) 역시 예상대로 결괏값을 보여 준다.

```
>>> 3 / 2.4
1.25
>>> 3 * 9
27
```

일반적으로 알고 있는 ÷나 ×가 아니라 키보드의 `/` (슬래시)와 `*` (별표, [ `Shift+8` ])를 사용한다는 것에 주의하자.

## 변수에 숫자 대입하고 계산하기

```
>>> a = 1
>>> b = 2
>>> a + b
3
```

a에 1, b에 2를 대입한 후 a와 b를 더하면 3이라는 결괏값을 보여 준다.

## 변수에 문자 대입하고 출력하기

```
>>> a = "Python"
>>> print(a)
Python
```

a 변수에 "Python"이라는 값을 대입한 후 print(a)라고 작성하면 a 값을 출력한다.

> *파이썬은 대소문자를 구별한다. print를 PRINT로 쓰면 정의되지 않았다는 오류 메시지가 나온다.*

print 함수 없이 변수 이름 a만 입력해도 a의 값을 확인할 수 있다.

```
>>> a = "Python"
>>> a
'Python'
```

> *`print(a)` 는 값만 깔끔하게 출력하고, 변수명 a만 입력하면 자료형을 알 수 있도록 따옴표가 포함되어 출력된다. 따옴표가 있으면 문자열이라는 뜻이다.*

## 조건문 if

다음은 조건문 if를 사용한 예제이다.

```
>>> a = 3
>>> if a > 1:
...     print("a는 1보다 큽니다.")
...
a는 1보다 큽니다.
```

이 예제는 a가 1보다 크면 "a는 1보다 큽니다."라는 문장을 출력(print)하라는 뜻이다. a는 3이므로 1보다 크다. 따라서 'a는 1보다 큽니다.'라는 문장이 출력된다.

> *인터프리터 창에서 프롬프트( `>>>` ) 자리에 나타나는 '...'은 아직 입력이 끝나지 않았다는 것을 의미한다.*

`if a > 1:` 다음 문장은 [ `Spacebar` ] 4개 또는 [ `Tab` ]을 이용해 반드시 들여쓰기한 후 print("a는 1보다 큽니다.")라고 작성해야 한다. 들여쓰기를 빠뜨리면 오류가 발생하는데, 이것은 파이썬 초보자가 가장 자주 만나는 실수이므로 주의하자. 들여쓰기 규칙은 '03장 제어문'에서 자세하게 알아본다. 바로 뒤에 이어지는 반복문 for, while 예제도 마찬가지로 들여쓰기가 필요하다.

> *실습 영상 : https://youtu.be/5CU-xTiMIZ0*

점프 투 파이썬

**최신 파이썬 버전의 자동 들여쓰기**

파이썬 3.10 이상 버전에서는 파이썬 셸에서 "..." 프롬프트가 나타난 후 자동으로 적절한 들여쓰기가 제공되어 [ `Tab` ]을 매번 입력하지 않아도 된다. 하지만 코드의 구조를 이해하기 위해 들여쓰기의 개념을 정확히 알고 있어야 한다.

## 반복문 for

다음은 for를 사용해서 [1, 2, 3] 안의 값을 하나씩 출력하는 예이다.

```
>>> for a in [1, 2, 3]:
...     print(a)
...
1
2
3
```

for 문을 사용하면 실행할 문장을 여러 번 반복할 수 있다. 위 예는 대괄호([]) 사이에 있는 값을 하나씩 출력한다. 이 코드의 의미는 '[1, 2, 3] 리스트의 앞에서부터 하나씩 꺼내 a 변수에 대입한 후 `print(a)` 를 수행하라'이다. 실행하면 a에 차례대로 1, 2, 3이라는 값이 대입되고 `print(a)` 에 의해 그 값이 차례대로 출력된다.

## 반복문 while

다음은 while을 사용하는 예이다.

```
>>> i = 0
>>> while i < 3:
...     i = i + 1
...     print(i)
...
1
2
3
```

while이라는 영어 단어의 뜻인 '~인 동안'에서 알 수 있듯이 while 문은 조건이 만족하는 동안 문장을 반복 수행한다. 위 예제는 i 값이 3보다 작은 동안 `i = i + 1`과 `print(i)`를 수행하라는 뜻이다. `i = i + 1`은 'i의 값을 1씩 더한다'라는 의미이다. i 값이 3보다 커지면 while 문을 빠져나간다.

> *실습 영상 : https://youtu.be/mtdbM3EOS68*

## 함수

파이썬에서 함수의 형태는 다음과 같다.

```
>>> def add(a, b):
...     return a + b
...
>>> add(3, 4)
7
```

def는 파이썬에서 함수를 정의할 때 사용하는 예약어이다. 위 예제는 add 함수를 만들고 그 함수를 어떻게 사용하는지 보여 준다. `add(a, b)`에서 a, b는 입력값, `a + b`는 결괏값이다. 즉 3, 4가 입력으로 들어오면 3 + 4를 수행하고 그 결괏값인 7을 반환한다.

> - *예약어란 프로그래밍 언어에서 이미 문법적인 용도로 사용하고 있는 단어를 말한다.*
> - *반환(return)은 함수에서 값을 돌려줄 때 사용한다.*

지금까지 기초적인 파이썬 문법에 대해 간략하게 알아보았다.

# 01-6 파이썬과 에디터

파이썬 대화형 인터프리터는 간단한 예제를 풀 때는 편리하지만, 여러 줄의 복잡한 코드를 작성할 때는 불편하다. 또한 인터프리터를 종료하면 작성한 프로그램이 사라져 다시 사용할 수 없다. 따라서 반복 사용할 프로그램을 만들 때는 에디터가 필요하다.

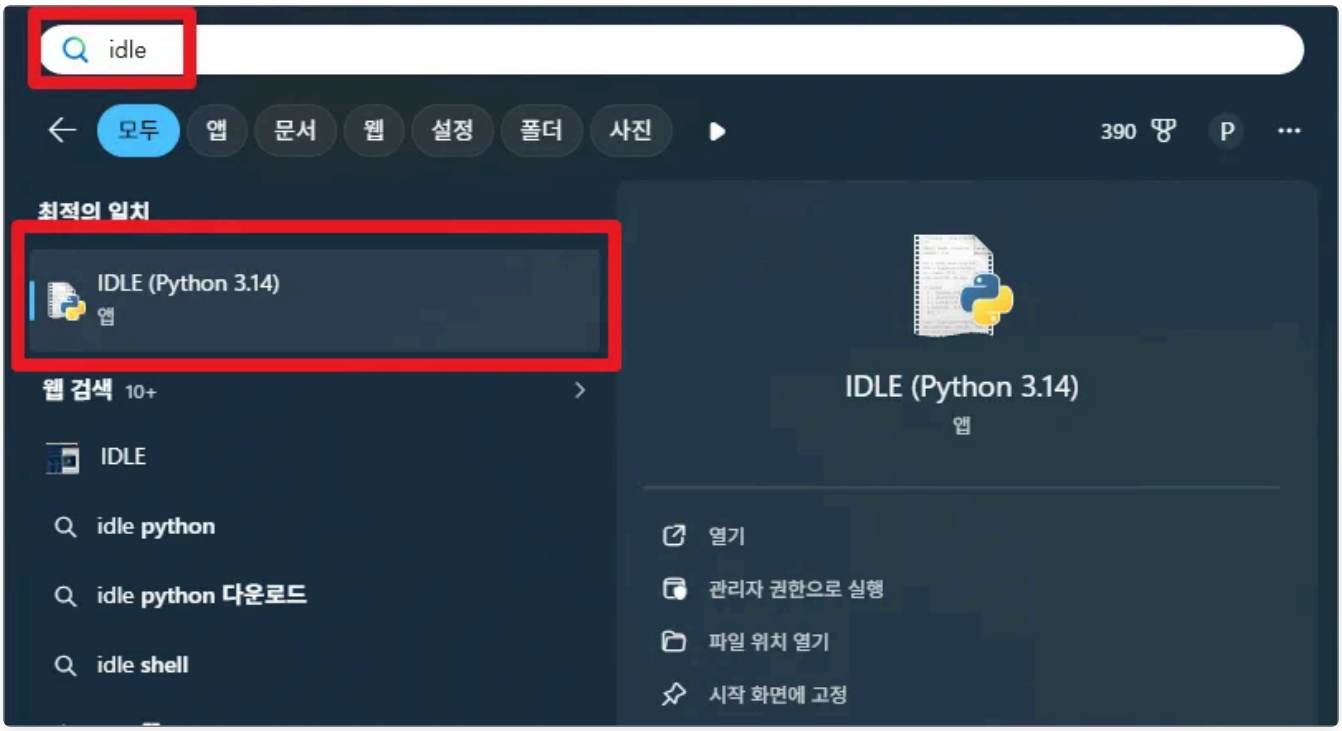
에디터(editor)란 소스 코드를 편집하는 프로그래밍 도구이다. 에디터에는 여러 종류가 있는데, 이 책에서는 파이썬 설치 시 기본으로 포함되는 파이썬 IDLE(Integrated Development and Learning Environment)의 에디터를 사용한다.

> *IDLE은 '아이들'이라고 읽는다.*


- IDLE 에디터로 파이썬 프로그램 작성하기
- 명령 프롬프트 창에서 파이썬 프로그램 실행하기
- 추천하는 에디터
    - 파이참
    - 비주얼 스튜디오 코드
        - 1. VS Code 설치하기
        - 2. 파이썬 extension 설치하기
        - 3. 작업 폴더 열기
        - 4. 파이썬 파일 작성하기
        - 5. 프로그램 실행하기

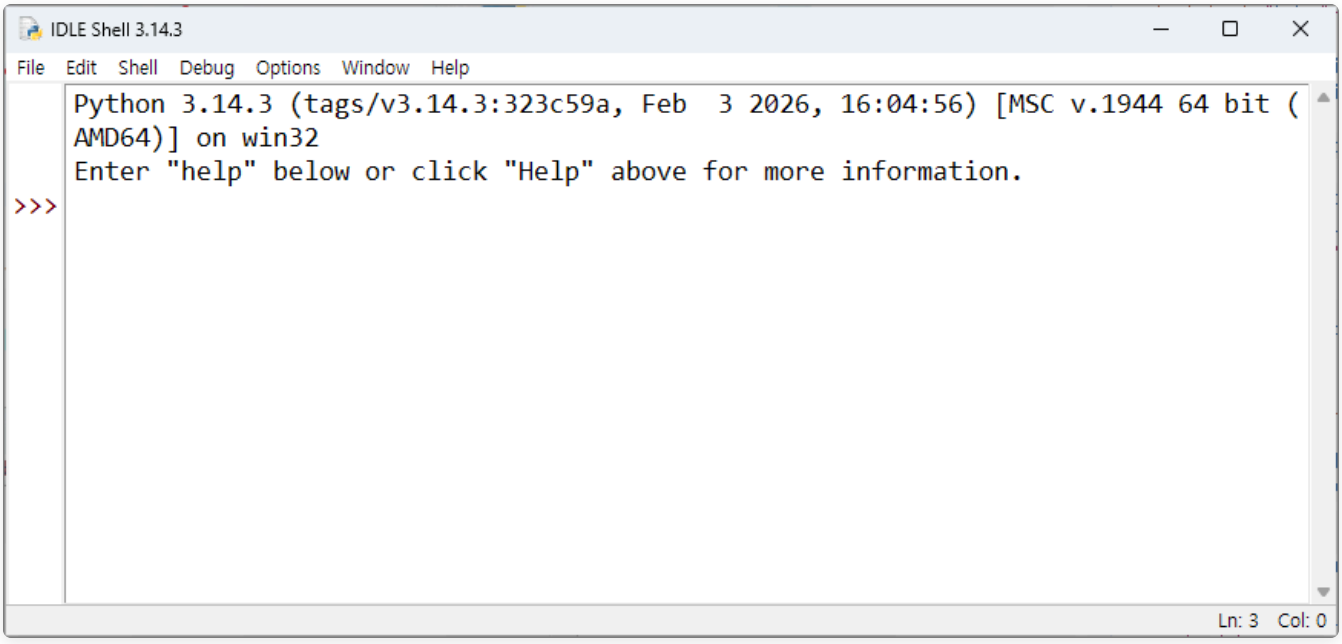

## IDLE 에디터로 파이썬 프로그램 작성하기

파이썬 IDLE은 파이썬 프로그램 작성을 도와주는 통합 개발 환경이다. [시작] 메뉴에서 'idle'을 검색한 후 파이썬 IDLE을 실행해 보자.




> *맥 사용자는 터미널(Spotlight에서 "터미널" 검색)을 열고 `idle3` 명령을 입력하자.*

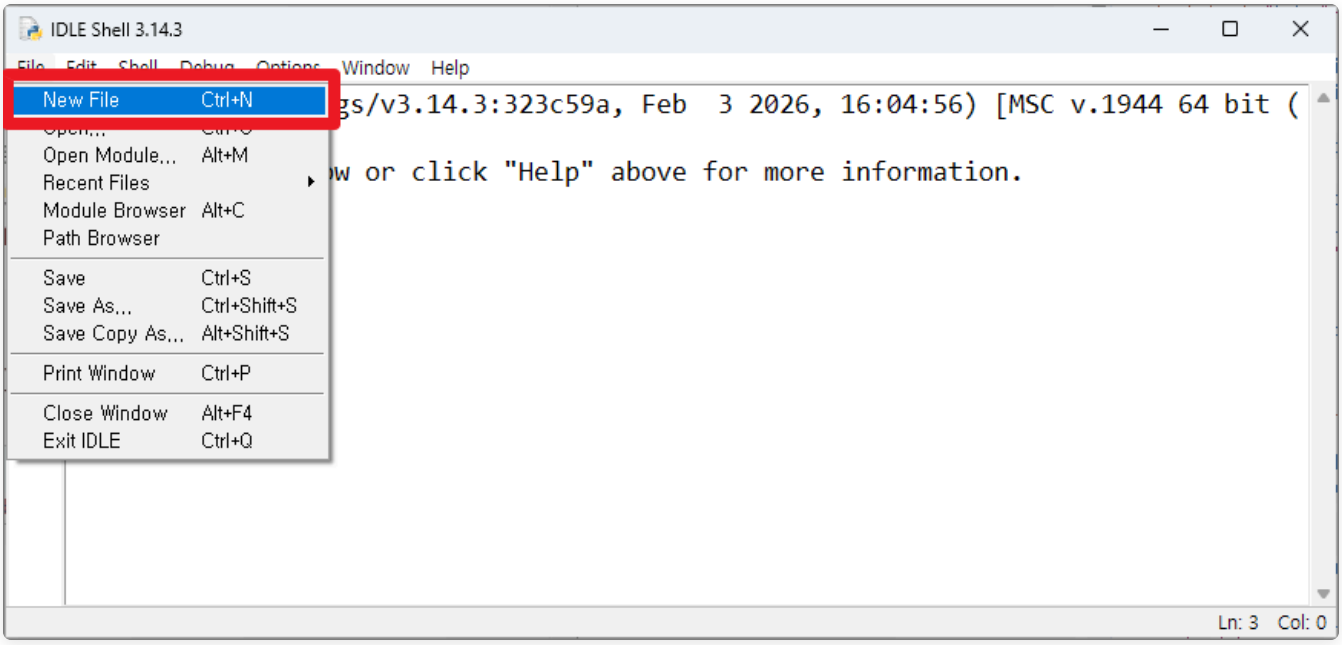
그러면 다음과 같은 IDLE 셸(shell) 창이 나타난다.



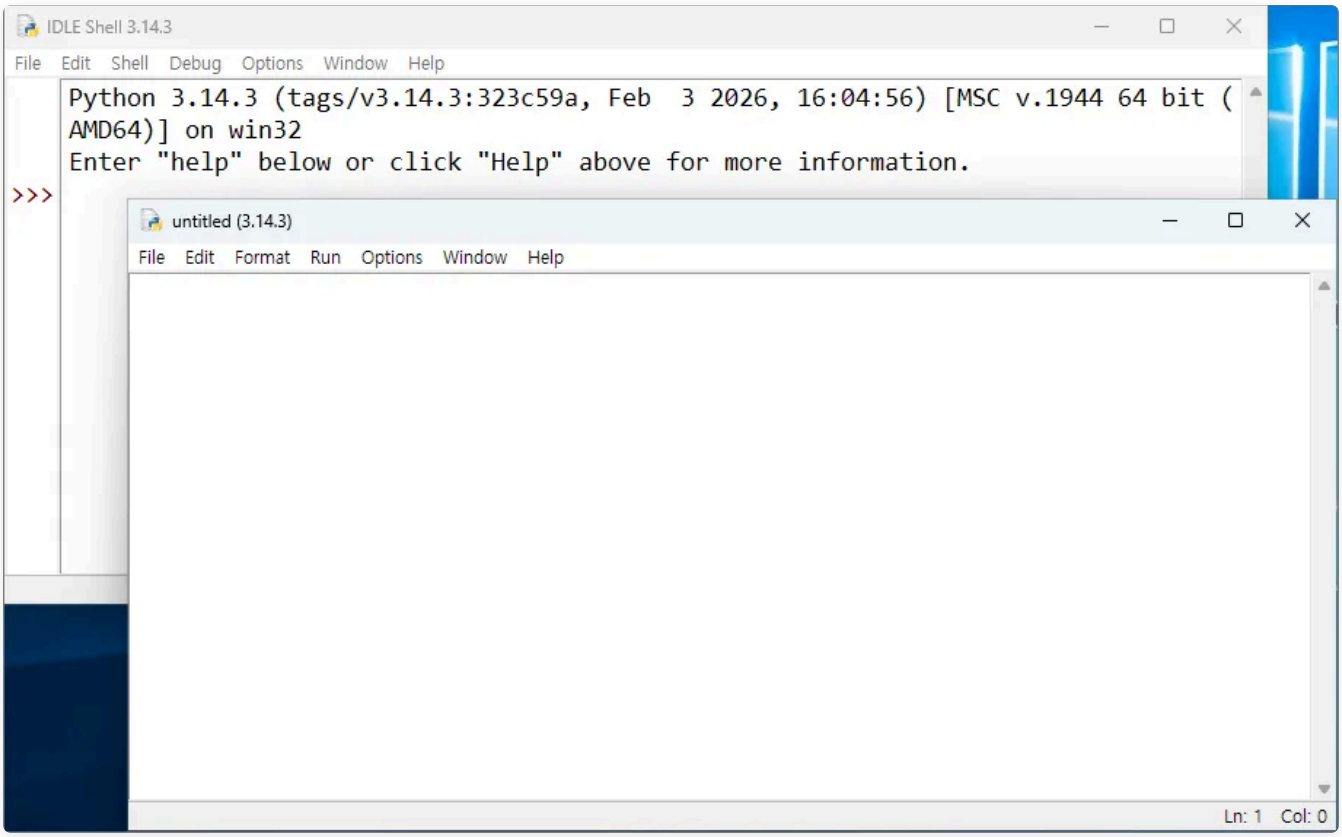

IDLE 셸은 IDLE 에디터에서 실행한 프로그램의 결과를 표시하는 창이다. 파이썬 코드를 직접 입력하고 실행할 수 있다는 점에서는 앞에서 살펴본 파이썬 셸과 같지만, 자동 완성이나 색상 표시 등의 기능이 추가되어 있어 동작이 조금 다를 수 있다. 이 책에서는 혼란을 방지하기 위해 IDLE 셸은 IDLE 에디터로 작성한 프로그램의 실행 결과를 표시하는 용도로만 사용한다. 즉, 프롬프트( `>>>` )로 시작하는 예제는 IDLE 셸이 아닌 명령 프롬프트(또는 터미널)의 파이썬 셸에서 실행해야 한다.

이제 IDLE 에디터를 실행해 보자.

IDLE 셸 창 메뉴에서 [File → New File]을 선택한다.

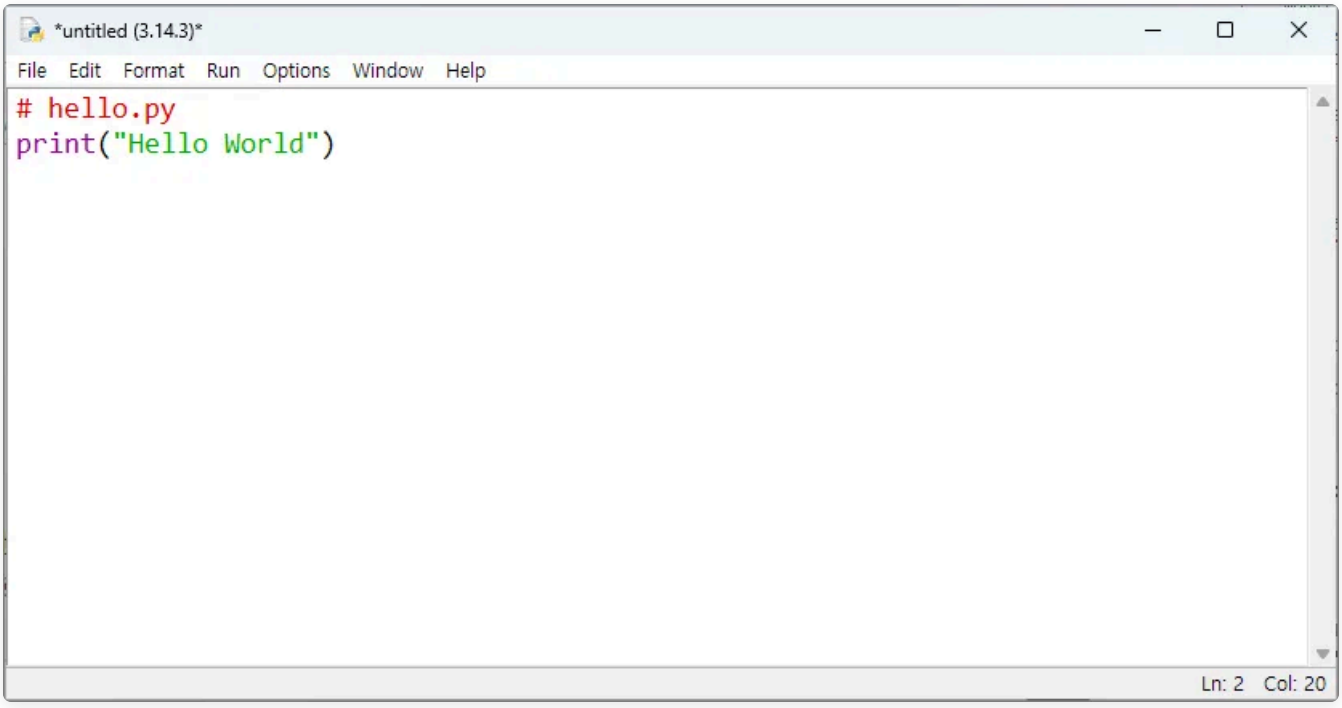



그러면 다음과 같이 빈 창이 나타나는데, 이것이 IDLE 에디터이다.




이제 IDLE 에디터에서 다음과 같이 파이썬 프로그램을 작성해 보자.

```
# hello.py
print("Hello World")
```




첫 줄의 `# hello.py` 는 주석이다. 파이썬에서 #으로 시작하는 문장은 주석으로 처리되어 프로그램 수행에 영향을 미치지 않는다. 주석은 소스 코드를 읽을 미래의 나 또는 다른 프로그래머를 위해 설명을 달 때 사용한다.

점프 투 파이썬

**여러 줄짜리 주석을 작성하는 방법**

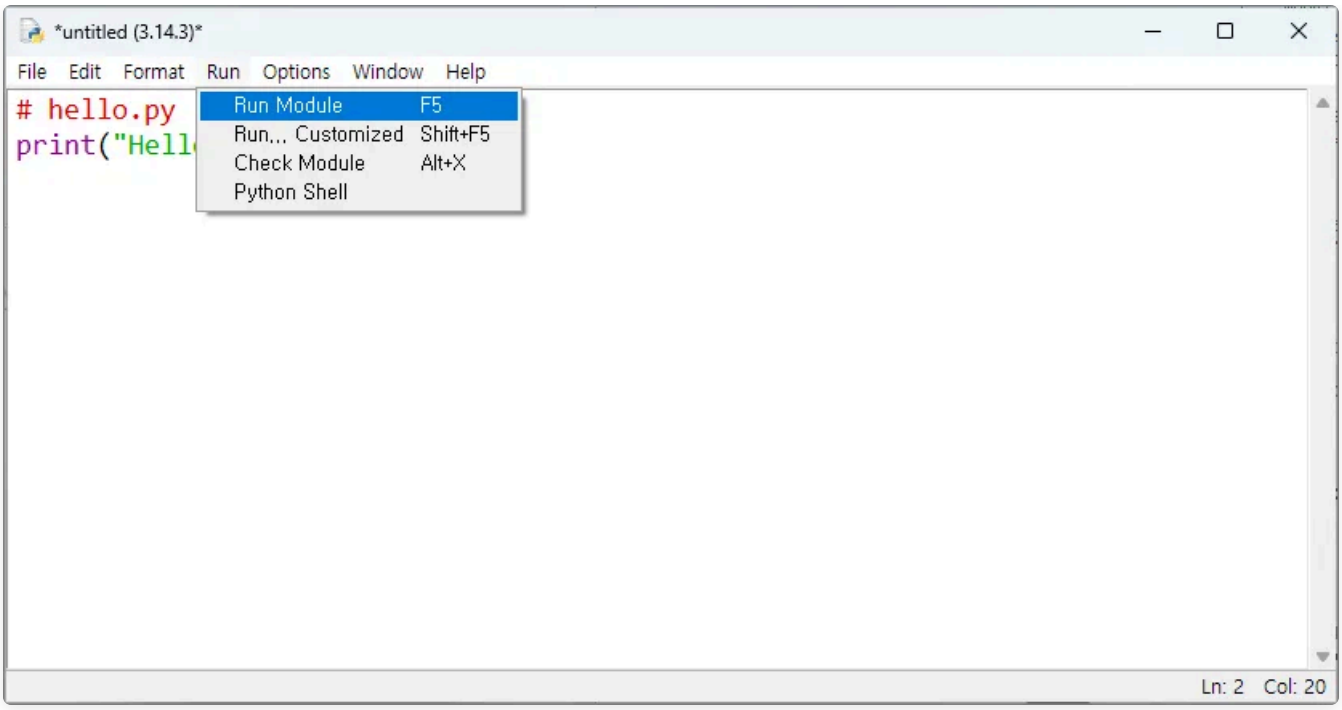
주석을 여러 줄 달고 싶다면 다음과 같이 작성하자.

```
"""
Author: EungYong Park
Date: 2023-05-01
이 프로그램은 Hello World를 출력하는 프로그램이다.
"""
```

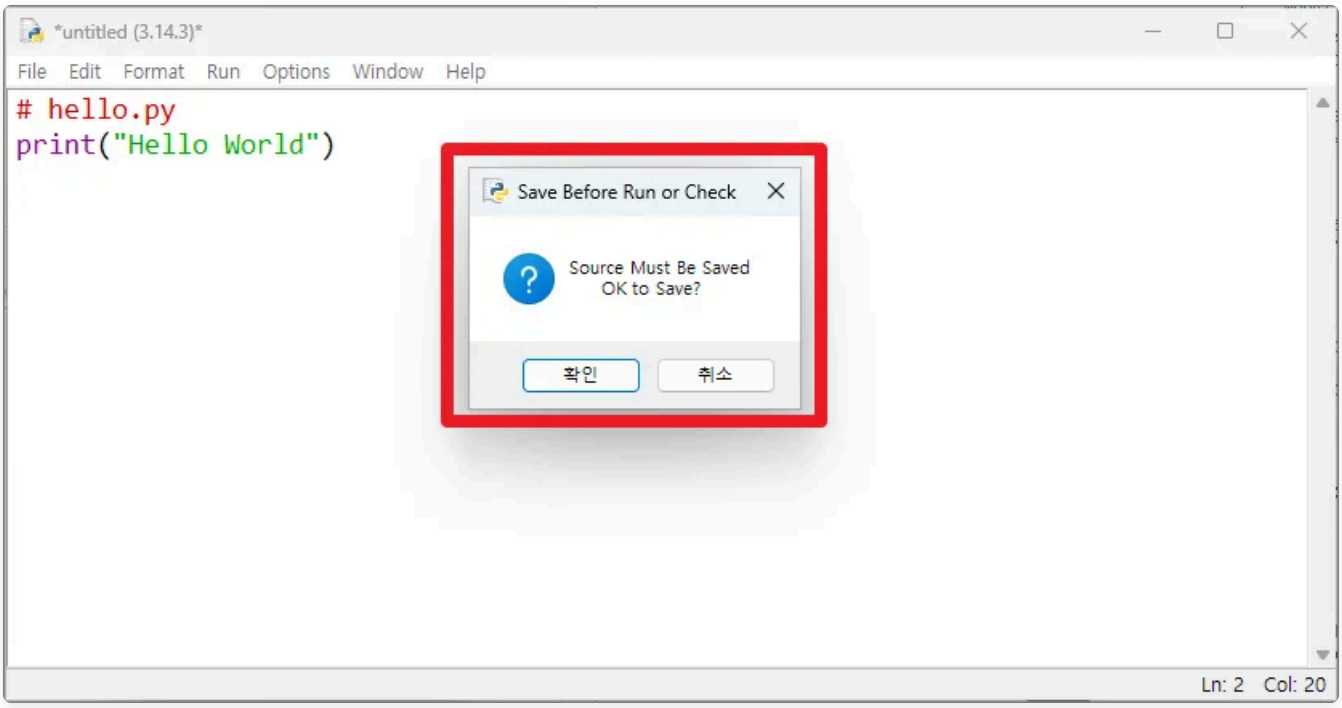
여러 줄 주석은 큰따옴표 3개를 연속으로 사용한 `"""` 사이에 작성하면 된다. 큰따옴표 대신 작은따옴표 3개( `'''` )를 사용해도 된다.

이제 작성한 프로그램을 실행해 보자.

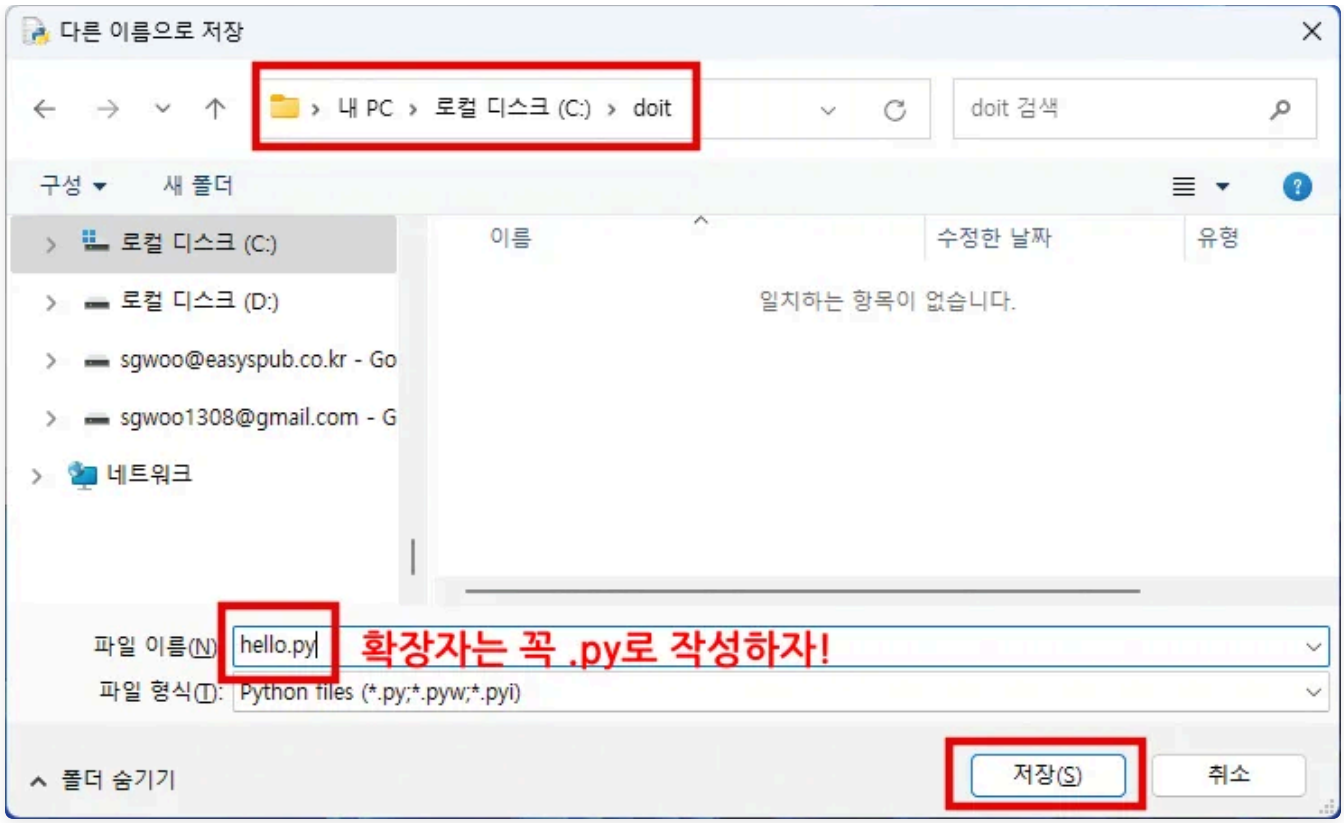
IDLE 에디터 창 메뉴에서 [Run → Run Module]을 선택한다(단축키: `F5` ).




프로그램을 실행하면 파일을 먼저 저장하라는 대화 상자가 나타난다.

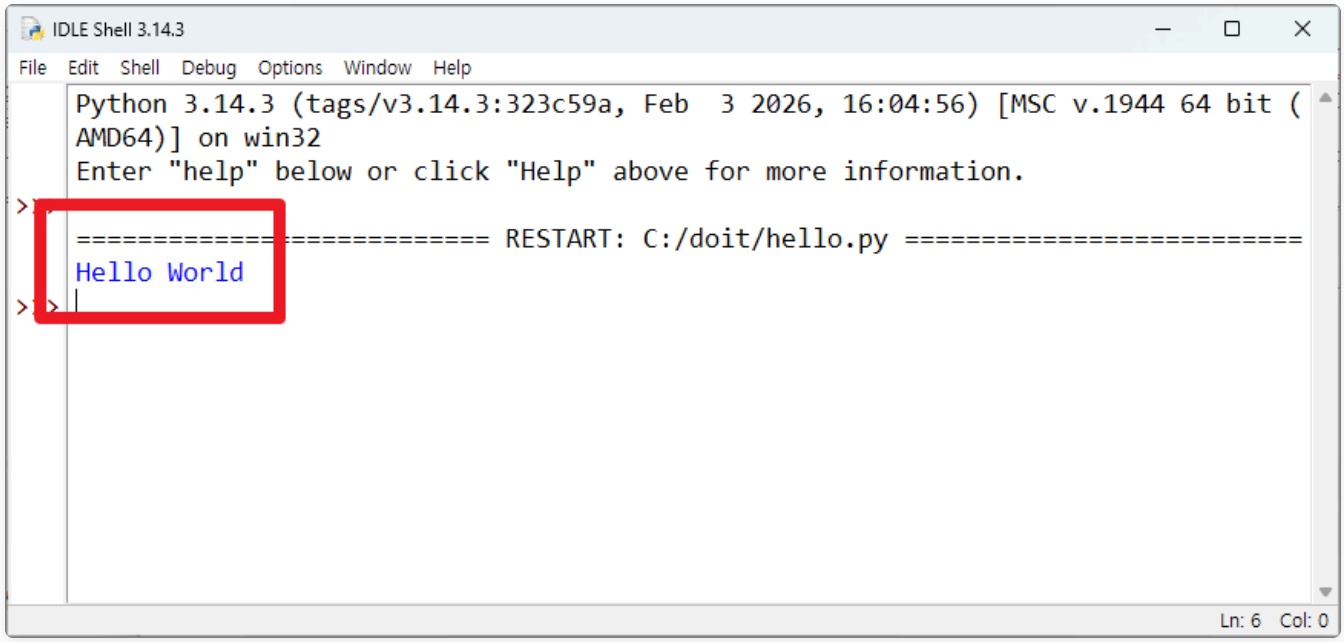



[확인]을 선택하면 [다른 이름으로 저장] 창이 나타난다. 로컬 디스크(C:)로 이동한 후 'doit'이라는 이름으로 디렉터리(폴더)를 생성하고 그 안에 hello.py라는 이름으로 파일을 저장하자. 에디터로 파이썬 프로그램을 작성한 후 저장할 때는 파일 이름의 확장자를 항상 .py로 해야 한다. .py는 파이썬 파일을 나타내는 확장자이다.




> *실습 경로를 간단히 하기 위해 `C:\doit` 디렉터리를 생성한 후 파일을 저장했다. `C:\` 에 파일을 바로 저장하면 오류가 발생할 수 있으므로 반드시 `C:\` 하위에 디렉터리를 생성한 후 그 안에 파일을 저장하자. 맥 사용자는 홈 디렉터리 아래에 `~/doit` 디렉터리를 생성하고 그 안에 파일을 저장하자.*

파일을 저장하면 hello.py 프로그램이 실행된다. 실행 결과는 다음과 같이 IDLE 셸 창에 표시된다.



앞으로 간단한 예제는 파이썬 셸에 입력해 결과를 바로 확인하고, 여러 줄을 한꺼번에 작성하거나 저장 후 재사용할 프로그램은 IDLE 에디터에서 작성한다.

점프 투 파이썬

**파이썬 셸과 IDLE 에디터의 차이점**

이 책은 파이썬 셸과 IDLE 에디터를 번갈아 사용하여 예제를 작성한다. 이때 주의할 점이 1가지 있다. 다음 예제를 살펴보자.

```
>>> a = "python"
>>> print(a)
python
>>> print(1 + 1)
2
```

파이썬 셸에서는 위와 같이 변수 a와 표현식 `1+1` 의 값을 확인하기 위해 print 함수를 사용해도 되지만, 다음처럼 print 함수 없이도 그 값을 확인할 수 있다.

```
>>> a = "python"
>>> a
'python'
>>> 1 + 1
2
```

이것은 표현식의 실행 결과를 출력하는 파이썬 셸 본래의 기능이기 때문이다. 하지만 IDLE 에디터를 사용할 때는 다음처럼 print 함수를 반드시 사용해야 한다.

```
# editor.py
a = "python"
print(a)
print(1 + 1)  # print 함수를 사용해야 결괏값을 출력할 수 있다.
```

실행 결과:

```
python
2
```

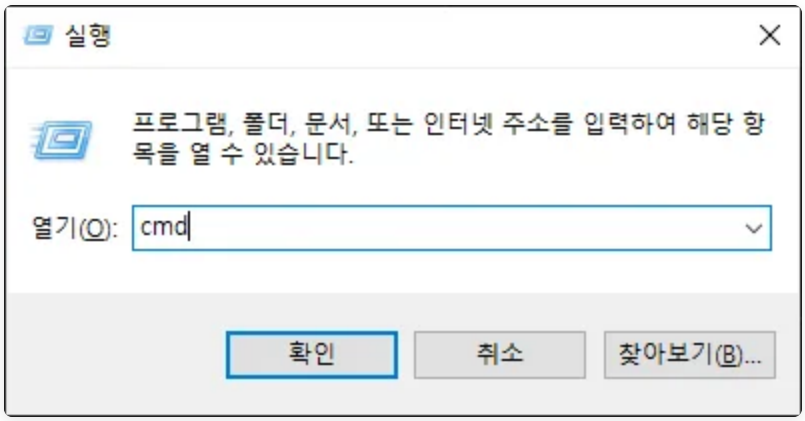
## 명령 프롬프트 창에서 파이썬 프로그램 실행하기

이번에는 앞에서 IDLE 에디터로 작성한 hello.py 파일을 명령 프롬프트(command prompt) 창에서 실행해 보자. IDLE 에디터에서 `F5`를 눌러 실행할 수도 있지만, 실제 업무에서는 주로 명령 프롬프트 창에서 파이썬 프로그램을 실행한다.

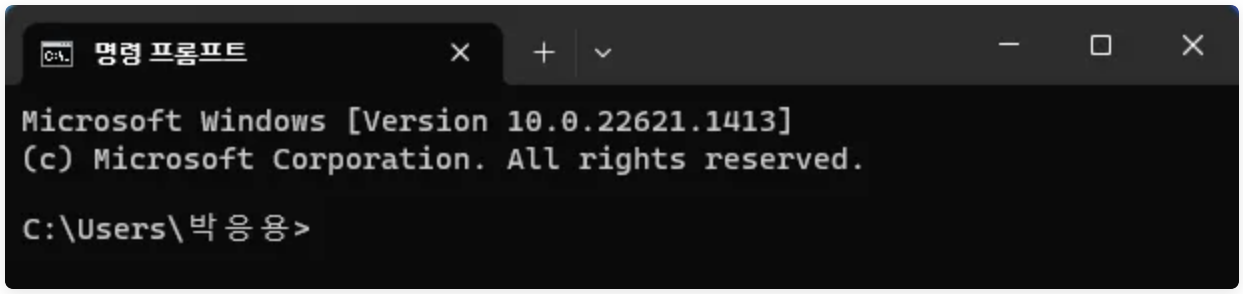
우리는 이미 다음 프로그램을 `C:\doit` 디렉터리에 'hello.py'라는 이름으로 저장했다.

```
# hello.py
print("Hello World")
```

hello.py 프로그램을 실행하기 위해 먼저 명령 프롬프트 창을 열어 보자. [`윈도우 키 + R`]을 누르면 다음과 같은 실행 창이 나타난다. 빈칸에 'cmd'를 입력하고 [확인]을 클릭하면 명령 프롬프트 창이 열린다.




첫 번째 줄에는 다음과 같은 프롬프트가 나타날 것이다. `C:\Users\박응용`에서 `박응용`은 필자의 컴퓨터 이름이다(여러분의 명령 프롬프트는 여러분의 컴퓨터 이름으로 표시될 것이다).




```
C:\Users\박응용>
```

> *맥 사용자는 명령 프롬프트 창 대신 터미널을 사용하자.*

이제 다음과 같이 프롬프트에서 `cd c:\doit` 이라고 입력한다. cd(change directory) 명령어는 프롬프트를 다음에 나오는 디렉터리로 이동시킨다.

```
C:\Users\박응용> cd c:\doit

c:\doit>
```

> *`c:\doit` 에서 역슬래시( `\` )는 [ `Enter` ] 바로 위에 있는 키로 입력한다. 한글 키보드에서는 원화 표시(₩)로 표시되어 있지만 같은 키이다.*

프롬프트가 `c:\doit>` 으로 바뀐 것을 확인할 수 있다. 이제 명령어 python과 실행할 프로그램 이름을 입력하면 파이썬 프로그램을 실행할 수 있다. 다음과 같이 입력해 hello.py 프로그램을 실행해 보자.

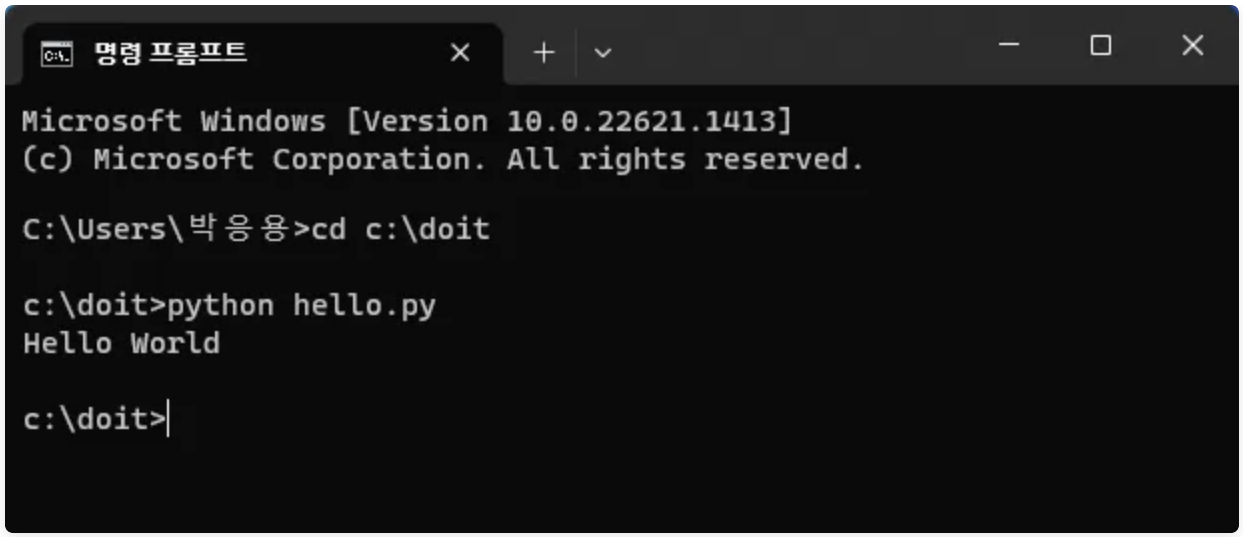
```
c:\doit> python hello.py
Hello World
```

맥 사용자는 터미널에서 다음처럼 실행하면 된다.

```
$ cd ~/doit
$ python3 hello.py
Hello World
```

hello.py 프로그램이 실행되는 것을 확인할 수 있다. 만약 결괏값이 위와 같지 않다면 hello.py 파일이 `C:\doit` 디렉터리에 있는지 다시 한번 확인하자.

명령 프롬프트 창에서 예제를 실행한 화면은 다음과 같다.




## 추천하는 에디터

파이썬으로 실무 프로그램을 작성할 때는 앞에서 소개한 IDLE보다 더 많은 기능을 갖춘 에디터를 사용하는 것이 좋다. 추천할 만한 에디터를 소개한다.

> *이 책의 예제를 풀 때는 파이썬 공부에 최적화된 IDLE을 사용하자. 여기서 소개하는 에디터는 파이썬 공부를 마친 후에 사용해 보기를 권장한다.*

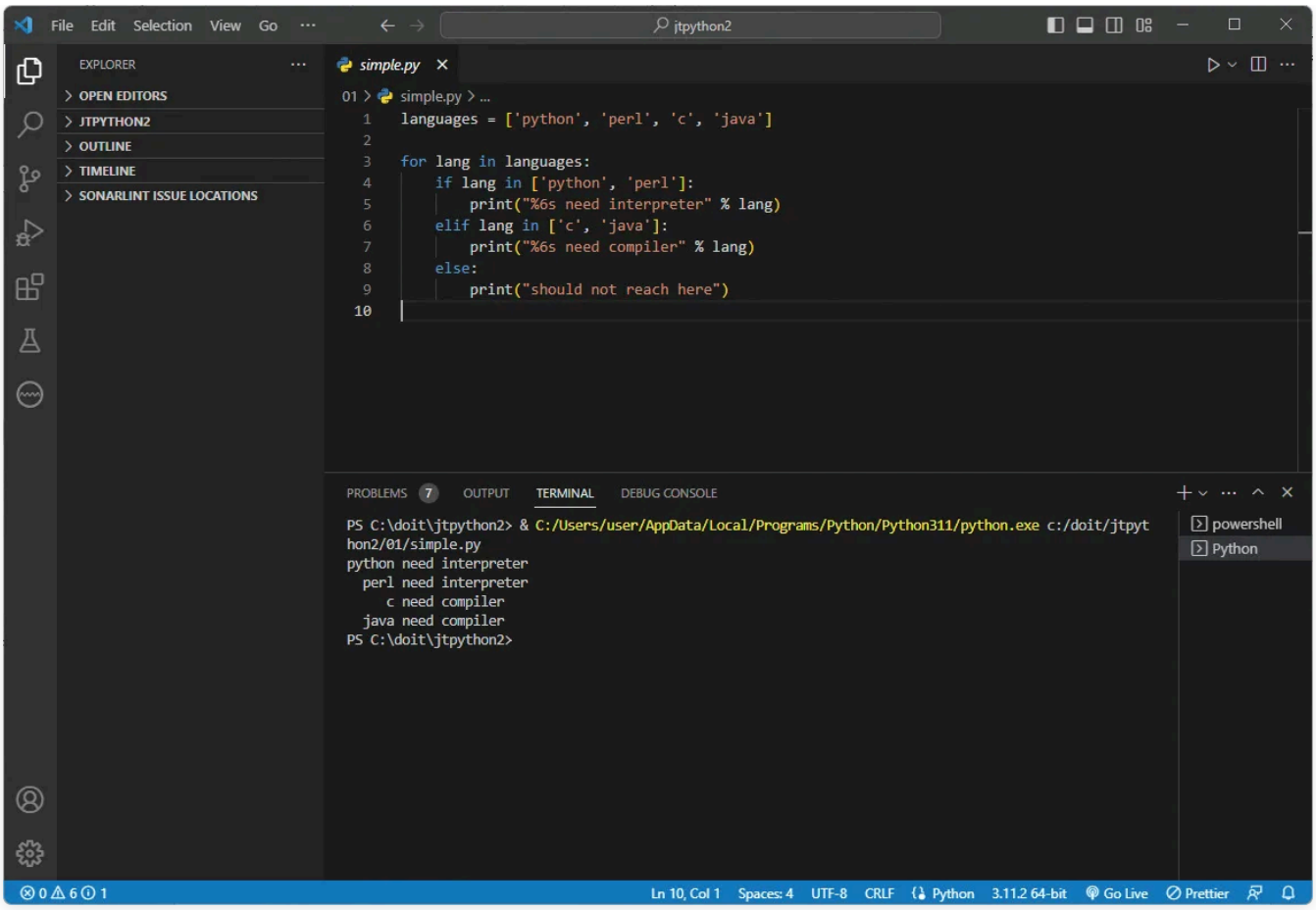
### 파이참

파이썬에 어느 정도 익숙해졌다면 파이참을 사용해 보기를 적극 추천한다. 파이참은 가장 유명한 파이썬 에디터 중 하나로, 코드 작성 시 자동 완성, 문법 체크 등 편리한 기능을 많이 제공한다. 공식 다운로드 사이트 (www.jetbrains.com/pycharm/download)에서 내려받을 수 있다.

파이참은 파이썬 전용 에디터이므로 별도의 extension이나 플러그인을 설치할 필요가 없다.

### 비주얼 스튜디오 코드

비주얼 스튜디오 코드(Visual Studio Code)는 파이참(PyCharm)과 더불어 프로그래머에게 가장 많은 사랑을 받는 파이썬 대표 에디터이다. 공식 다운로드 사이트(code.visualstudio.com)에서 내려받을 수 있다.




비주얼 스튜디오 코드는 파이썬 전용 에디터가 아니므로 파이썬 외에도 여러 언어를 지원한다. 따라서 비주얼 스튜디오 코드를 설치한 후 파이썬을 사용하려면 가장 먼저 파이썬 extension을 설치해야 한다. 파이썬 extension은 비주얼 스튜디오 코드의 [Extensions] 메뉴에서 설치할 수 있다.

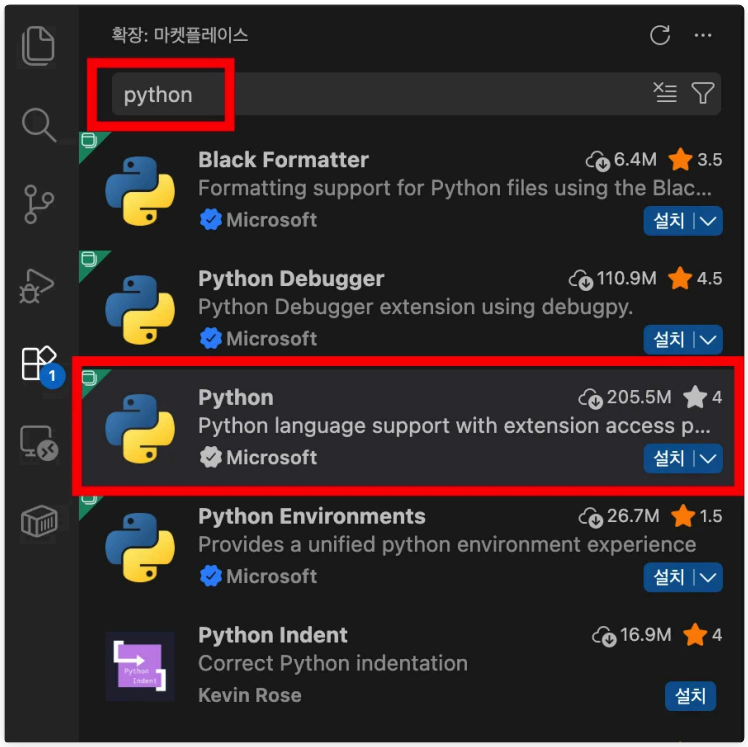
여기서는 비주얼 스튜디오 코드(이하 VS Code)로 파이썬 프로그램을 작성하고 실행하는 방법을 간단히 알아보자.

### 1. VS Code 설치하기

공식 다운로드 사이트(code.visualstudio.com)에서 VS Code를 내려받아 설치한다.

### 2. 파이썬 extension 설치하기

VS Code를 실행한 후 왼쪽 사이드바에서 Extensions 아이콘(블록 모양)을 클릭한다. 검색창에 'python'을 입력하면 Microsoft에서 제공하는 Python extension이 나타난다. [설치]를 클릭하여 설치하자.




이 extension을 설치하면 코드 자동 완성, 문법 강조, 오류 표시 등 파이썬 개발에 필요한 기능을 사용할 수 있다.

### 3. 작업 폴더 열기

VS Code 메뉴에서 [File → Open Folder]를 선택하고, 앞에서 만든 `C:\doit` 폴더를 선택한다.

### 4. 파이썬 파일 작성하기

왼쪽 탐색기(Explorer)에서 새 파일 아이콘을 클릭하고 `hello_vscode.py` 라는 이름으로 파일을 만든다. 그리고 다음과 같이 코드를 작성한 후 `[Ctrl+S]` (맥: `[Cmd+S]` )를 눌러 저장하자.

```
# hello_vscode.py
print("Hello VS Code!")
```

### 5. 프로그램 실행하기

에디터 오른쪽 상단의 실행 버튼(▷)을 클릭하면 하단의 터미널 창에서 프로그램이 실행된다.




터미널 창에 다음과 같이 출력되면 성공이다.

```
Hello VS Code!
```

> *또는 VS Code 하단의 터미널 창에서 `python hello_vscode.py` 를 직접 입력해도 같은 결과를 얻을 수 있다.*